

STAR RX72N - imu_test Bench Test
1.0.0

Generated by Doxygen 1.16.1

1 Directory Hierarchy	1
1.1 Directories	1
2 File Index	3
2.1 File List	3
3 Directory Documentation	5
3.1 imu_test Directory Reference	5
4 File Documentation	7
4.1 imu_test/clock.c File Reference	7
4.1.1 Detailed Description	8
4.1.2 Enumeration Type Documentation	8
4.1.2.1 clock_byte_constants_t	8
4.1.2.2 clock_extra_addrs_t	9
4.1.2.3 clock_sckcr_t	9
4.1.2.4 clock_timeout_t	9
4.1.2.5 clock_word_constants_t	10
4.1.3 Function Documentation	10
4.1.3.1 clock_init()	10
4.2 clock.c	11
4.3 imu_test/linker.ld File Reference	13
4.4 linker.ld	13
4.5 imu_test/main.c File Reference	14
4.5.1 Detailed Description	16
4.5.2 Macro Definition Documentation	16
4.5.2.1 I16_LE	16
4.5.2.2 REG16	16
4.5.2.3 REG32	17
4.5.2.4 REG8	17
4.5.3 Enumeration Type Documentation	17
4.5.3.1 delay_const_t	17
4.5.3.2 gpio_pins_t	17
4.5.3.3 poll_max_t	18
4.5.3.4 port_base_t	18
4.5.3.5 riic_addrs_t	18
4.5.3.6 riic_bits_t	19
4.5.3.7 riic_off_t	20
4.5.3.8 sensor_t	20
4.5.4 Function Documentation	21
4.5.4.1 busy_wait_ms()	21
4.5.4.2 clock_init()	22
4.5.4.3 gpio_output()	23
4.5.4.4 gpio_set()	23

4.5.4.5 i2c_bus_recover()	24
4.5.4.6 i2c_read_byte()	24
4.5.4.7 i2c_read_bytes()	26
4.5.4.8 i2c_write_byte()	27
4.5.4.9 imu_reset_pulse()	29
4.5.4.10 main()	29
4.5.4.11 r1()	32
4.5.4.12 riic1_init()	33
4.5.4.13 riic1_recover()	34
4.5.4.14 sci9_debug_init()	35
4.5.4.15 sci9_debug_puthex16()	36
4.5.4.16 sci9_debug_putint16_col()	37
4.5.4.17 sci9_debug_puts()	37
4.5.4.18 wait_bit()	38
4.5.5 Variable Documentation	39
4.5.5.1 s_last_fail_gate	39
4.5.5.2 s_last_write_gate	39
4.6 main.c	39
4.7 imu_test/sci9.c File Reference	47
4.7.1 Enumeration Type Documentation	48
4.7.1.1 hex_shifts_t	48
4.7.1.2 sci9_addrs_t	48
4.7.1.3 sci9_cfg_t	49
4.7.1.4 sci9_settle_t	49
4.7.2 Function Documentation	50
4.7.2.1 sci9_brr()	50
4.7.2.2 sci9_debug_init()	50
4.7.2.3 sci9_debug_putc()	51
4.7.2.4 sci9_debug_puthex16()	52
4.7.2.5 sci9_debug_puthex32()	52
4.7.2.6 sci9_debug_puts()	53
4.7.2.7 sci9_scmr()	53
4.7.2.8 sci9_scr()	54
4.7.2.9 sci9_semr()	54
4.7.2.10 sci9_smr()	54
4.7.2.11 sci9_ssr()	55
4.7.2.12 sci9_tdr()	55
4.8 sci9.c	56
4.9 imu_test/stubs.c File Reference	58
4.9.1 Detailed Description	59
4.9.2 Function Documentation	59
4.9.2.1 rx_log_uart_putc()	59
4.9.2.2 rx_log_uart_puthex()	59

4.9.2.3 rx_log_uart_putint()	59
4.9.2.4 rx_log_uart_puts()	59
4.9.2.5 rx_log_uart_putuint()	60
4.9.2.6 uart_debug_putc()	60
4.9.2.7 uart_debug_puthex()	60
4.9.2.8 uart_debug_putint()	60
4.9.2.9 uart_debug_puts()	60
4.9.2.10 uart_debug_putuint()	60
4.10 stubs.c	61

Chapter 1

Directory Hierarchy

1.1 Directories

imu_test	5
clock.c	7
linker.ld	13
main.c	14
sci9.c	47
stubs.c	58

Chapter 2

File Index

2.1 File List

Here is a list of all files with brief descriptions:

imu_test/ clock.c	
RX72N clock init for the production STAR PCB: MOSC 24 MHz crystal -> PLL 240 MHz -> ICLK=240 / PCKA=120 / PCKB=60 / FCLK=60 MHz	7
imu_test/ linker.ld	13
imu_test/ main.c	
BNO055 + BMP280 over RIIC1 smoke test on the production STAR PCB	14
imu_test/ sci9.c	47
imu_test/ stubs.c	
Log/UART no-op stubs for the bare-metal IMU smoke test	58

Chapter 3

Directory Documentation

3.1 imu_test Directory Reference

Files

- file [clock.c](#)
RX72N clock init for the production STAR PCB: MOSC 24 MHz crystal -> PLL 240 MHz -> ICLK=240 / PCKA=120 / PCKB=60 / FCLK=60 MHz.
- file [linker.ld](#)
- file [main.c](#)
BNO055 + BMP280 over RIIC1 smoke test on the production STAR PCB.
- file [sci9.c](#)
- file [stubs.c](#)
Log/UART no-op stubs for the bare-metal IMU smoke test.

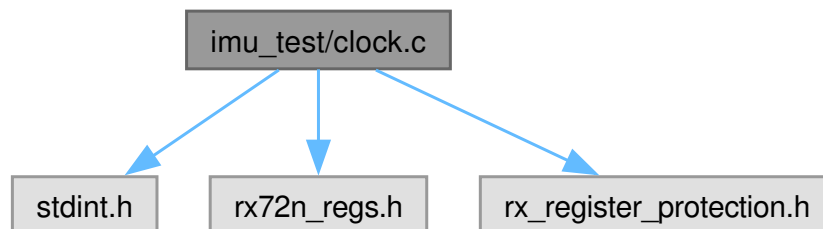
Chapter 4

File Documentation

4.1 imu_test/clock.c File Reference

RX72N clock init for the production STAR PCB: MOSC 24 MHz crystal -> PLL 240 MHz -> ICLK=240 / PCKA=120 / PCKB=60 / FCLK=60 MHz.

```
#include <stdint.h>
#include "rx72n_regs.h"
#include "rx_register_protection.h"
Include dependency graph for clock.c:
```



Enumerations

- enum `clock_extra_addrs_t` : `uintptr_t` { `k_packcr_addr` = 0x00080044U }
- enum `clock_word_constants_t` : `uint16_t` { `k_pllcr_mosc_x10` = 0x1300U }
- enum `clock_sckcr_t` : `uint32_t` { `k_sckcr_value` = 0x21C21211U }
- enum `clock_byte_constants_t` : `uint8_t` {
 `k_sub_clock_stopped` = 0x01U , `k_main_osc_enabled` = 0x00U , `k_pll_enabled` = 0x00U ,
 `k_oscovfsr_moovf` = 0x01U ,
 `k_oscovfsr_plovf` = 0x04U }
- enum `clock_timeout_t` : `uint32_t` { `k_main_osc_poll_max` = 500000U , `k_pll_lock_poll_max` = 500000U }

Functions

- void `clock_init` (void)

4.1.1 Detailed Description

RX72N clock init for the production STAR PCB: MOSC 24 MHz crystal -> PLL 240 MHz -> ICLK=240 / PCKA=120 / PCKB=60 / FCLK=60 MHz.

Self-contained equivalent of `src/rx_clock_power_init.c`, stripped of the logging, asserts, simulator branches, and PLL/USB clock paths the motor spin test does not need. Configures only the main PLL.

Path: EXTAL 24 MHz -> MOSC -> PLL ($\times 10 / 1$) = 240 MHz -> SCKCR=0x21C21211. After this routine returns, ICLK is 240 MHz and PCLKA (the GPTW source) is 120 MHz, matching `k_pclka_hz` in `libs/rx_hal/inc/rx72n_clock.h`.

SPDX-License-Identifier: MIT

Copyright

Copyright (c) 2026 Locked Inc.

Definition in file [clock.c](#).

4.1.2 Enumeration Type Documentation

4.1.2.1 `clock_byte_constants_t`

enum [clock_byte_constants_t](#) : `uint8_t`

Enumerator

<code>k_sub_clock_stopped</code>	SOSCCR: stop sub-clock oscillator
<code>k_main_osc_enabled</code>	MOSCCR: 0 = MOSC running
<code>k_pll_enabled</code>	PLLCR2: 0 = PLL running
<code>k_oscovfsr_moovf</code>	OSCOVFSR bit 0: MOSC stable
<code>k_oscovfsr_plovf</code>	OSCOVFSR bit 2: PLL stable

Definition at line 58 of file [clock.c](#).

```
00058      : uint8_t {
00059  k_sub_clock_stopped = 0x01U, /**< SOSCCR: stop sub-clock oscillator */
00060  k_main_osc_enabled  = 0x00U, /**< MOSCCR: 0 = MOSC running */
00061  k_pll_enabled       = 0x00U, /**< PLLCR2: 0 = PLL running */
00062  k_oscovfsr_moovf    = 0x01U, /**< OSCOVFSR bit 0: MOSC stable */
00063  k_oscovfsr_plovf    = 0x04U, /**< OSCOVFSR bit 2: PLL stable */
00064 } clock_byte_constants_t;
```

4.1.2.2 clock_extra_addrs_t

```
enum clock_extra_addrs_t : uintptr_t
```

Enumerator

k_packcr_addr	Peripheral clock source: 0=PLL
---------------	--------------------------------

Definition at line 27 of file [clock.c](#).

```
00027     : uintptr_t {
00028   k_packcr_addr = 0x00080044U, /**< Peripheral clock source: 0=PLL */
00029 } clock_extra_addrs_t;
```

4.1.2.3 clock_sckcr_t

```
enum clock_sckcr_t : uint32_t
```

Enumerator

k_sckcr_value	
---------------	--

Definition at line 49 of file [clock.c](#).

```
00049     : uint32_t {
00050   /*
00051    * SCKCR dividers (matches production k_system_clock_dividers):
00052    *   FCK=/4 (60), ICK=/1 (240), PSTOP1=1, PSTOP0=1, BCK=/4 (60),
00053    *   PCKA=/2 (120), PCKB=/4 (60), PCKC=/4 (60), PCKD=/4 (60).
00054    */
00055   k_sckcr_value = 0x21C21211U,
00056 } clock_sckcr_t;
```

4.1.2.4 clock_timeout_t

```
enum clock_timeout_t : uint32_t
```

Enumerator

k_main_osc_poll_max	
k_pll_lock_poll_max	

Definition at line 66 of file [clock.c](#).

```
00066     : uint32_t {
00067   /* Bounded poll counts (NASA Rule 2). We poll the hardware ready flag
00068    * (OSCOVFSR.MOOVF / .PLOVF) so the actual wait is hardware-determined --
00069    * roughly 10 ms physical time regardless of CPU clock. The upper bound
00070    * just prevents an infinite spin if the crystal is missing. ~500k iters
00071    * at LOCO 240 kHz is ~3 s, which is still much shorter than the prior
00072    * fixed 10-second busy-wait. */
00073   k_main_osc_poll_max = 500000U,
00074   k_pll_lock_poll_max = 500000U,
00075 } clock_timeout_t;
```

4.1.2.5 clock_word_constants_t

enum `clock_word_constants_t` : `uint16_t`

Enumerator

k_pllcr_mosc_x10	
------------------	--

Definition at line 34 of file `clock.c`.

```

00034         : uint16_t {
00035     /*
00036     * PLLCR layout:
00037     * bits[1:0] PLIDIV : 00 = /1
00038     * bit [4]  PLLSRCSEL : 0 = MOSC, 1 = HOCO
00039     * bits[13:8] STC      : multiplier = (STC + 1) / 2
00040     *
00041     * MOSC 24 MHz / 1 * 10 = 240 MHz. STC = (10 * 2) - 1 = 19 = 0x13.
00042     * PLLSRCSEL = 0 (MOSC) -> bit 4 clear.
00043     *
00044     * Production rx_clock_power_init.c uses k_pll_multiplier_10_div_1 = 0x1300.
00045     */
00046     k_pllcr_mosc_x10 = 0x1300U,
00047 } clock_word_constants_t;

```

4.1.3 Function Documentation

4.1.3.1 clock_init()

void `clock_init` (
 void)

Definition at line 80 of file `clock.c`.

```

00081 {
00082     volatile rx_system_regs_t *sys = system_regs();
00083
00084     /* Unlock PRC0 (clock) + PRC1 (module stop) write protection. */
00085     *prcr_reg() = k_rx_prcr_unlock_prc0_prc1;
00086
00087     /* Stop sub-clock oscillator -- not used on STAR. */
00088     sys->sosccr = k_sub_clock_stopped;
00089
00090     /* Start MOSC (24 MHz external crystal). */
00091     sys->mosccr = k_main_osc_enabled;
00092
00093     /* Poll the hardware MOSC-stable flag. Exits as soon as the crystal has
00094     * settled (typically ~10 ms physical time) instead of waiting a fixed
00095     * 2.4M cycles, which on the LOCO 240 kHz boot clock is ~10 seconds. */
00096     for (volatile uint32_t i = 0; i < k_main_osc_poll_max; i++) {
00097         if ((sys->oscofsr & k_oscofsr_moovf) != 0U) {
00098             break;
00099         }
00100     }
00101
00102     /* Configure and enable PLL: 24 MHz x 10 / 1 = 240 MHz. */
00103     sys->pllcr = k_pllcr_mosc_x10;
00104     sys->pllcr2 = k_pll_enabled;
00105
00106     /* Wait for PLL lock (bounded). If we time out, the crystal is missing
00107     * or not driving -- DO NOT switch SKCR3 to PLL or the chip will halt
00108     * on a dead clock. Stay on LOCO so the LEDs still respond and the user
00109     * can see we got past clock_init. */
00110     bool pll_locked = false;
00111     for (volatile uint32_t i = 0; i < k_pll_lock_poll_max; i++) {
00112         if ((sys->oscofsr & k_oscofsr_plovf) != 0U) {
00113             pll_locked = true;
00114             break;
00115         }
00116     }
00117
00118     if (pll_locked) {
00119         /* MANDATORY: raise flash wait state BEFORE switching to >120 MHz. */
00120         *memwait_reg() = k_memwait_one_wait;
00121     }

```



```

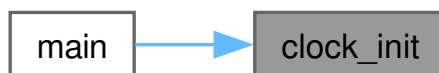
00122      /* Apply system clock dividers. */
00123      sys->sckcr = k_sckcr_value;
00124
00125      /* Route UCLK from main PLL (UPLLSEL=0). */
00126      *((volatile uint16_t *)k_packcr_addr) = 0x0000U;
00127
00128      /* Switch system clock source to PLL. */
00129      sys->sckcr3 = k_sckcr3_cksel_pll;
00130  }
00131
00132  /* Re-lock PRCR. */
00133  *prcr_reg() = k_rx_prcr_lock;
00134  }

```

References `k_main_osc_enabled`, `k_main_osc_poll_max`, `k_oscvsfr_moovf`, `k_oscvsfr_plovf`, `k_packer_addr`, `k_pll_enabled`, `k_pll_lock_poll_max`, `k_pllcr_mosc_x10`, `k_sckcr_value`, and `k_sub_clock_stopped`.

Referenced by [main\(\)](#).

Here is the caller graph for this function:



4.2 clock.c

Go to the documentation of this file.

```

00001  /*
00002  * @file clock.c
00003  * @brief RX72N clock init for the production STAR PCB: MOSC 24 MHz crystal
00004  *      -> PLL 240 MHz -> ICLK=240 / PCKA=120 / PCKB=60 / FCLK=60 MHz.
00005  *
00006  * @details
00007  * Self-contained equivalent of src/rx_clock_power_init.c, stripped of the
00008  * logging, asserts, simulator branches, and PPLL/USB clock paths the motor
00009  * spin test does not need. Configures only the main PLL.
00010  *
00011  * Path: EXTAL 24 MHz -> MOSC -> PLL (x10 / 1) = 240 MHz -> SCKCR=0x21C21211.
00012  * After this routine returns, ICLK is 240 MHz and PCLKA (the GPTW source)
00013  * is 120 MHz, matching k_pclka_hz in libs/rx_hal/inc/rx72n_clock.h.
00014  *
00015  * SPDX-License-Identifier: MIT
00016  * @copyright Copyright (c) 2026 Locked Inc.
00017  */
00018
00019 #include <stdint.h>
00020
00021 #include "rx72n_regs.h"
00022 #include "rx_register_protection.h"
00023
00024 /*
=====
00025  * Register addresses not exposed by rx_system_regs_t
00026  *
=====
00027  */
00028 typedef enum : uintptr_t {
00029     k_packcr_addr = 0x00080044U, /**< Peripheral clock source: 0=PLL */
00029 } clock_extra_addrs_t;
00030
00031 /*
=====
00032  * Bit / value constants (mirrors src/rx_clock_power_init.c)
00033  *
=====
00034  */
00034 typedef enum : uint16_t {
00035     /**
00036      * PLLCR layout:
00037      * bits[1:0] PLIDIV : 00 = /1
00038      * bit [4] PLLSRCSEL : 0 = MOSC, 1 = HOCO
00039      * bits[13:8] STC : multiplier = (STC + 1) / 2

```

```

00040  *
00041  * MOSC 24 MHz / 1 * 10 = 240 MHz. STC = (10 * 2) - 1 = 19 = 0x13.
00042  * PLLSRCSEL = 0 (MOSC) -> bit 4 clear.
00043  *
00044  * Production rx_clock_power_init.c uses k_pll_multiplier_10_div_1 = 0x1300.
00045  */
00046  k_pllcr_mosc_x10 = 0x1300U,
00047 } clock_word_constants_t;
00048
00049 typedef enum : uint32_t {
00050  /*
00051  * SCKCR dividers (matches production k_system_clock_dividers):
00052  *   FCK=/4 (60), ICK=/1 (240), PSTOP1=1, PSTOP0=1, BCK=/4 (60),
00053  *   PCKA=/2 (120), PCKB=/4 (60), PCKC=/4 (60), PCKD=/4 (60).
00054  */
00055  k_sckcr_value = 0x21C21211U,
00056 } clock_sckcr_t;
00057
00058 typedef enum : uint8_t {
00059  k_sub_clock_stopped = 0x01U, /**< SOSCCR: stop sub-clock oscillator */
00060  k_main_osc_enabled = 0x00U, /**< MOSCCR: 0 = MOSC running */
00061  k_pll_enabled = 0x00U, /**< PLLCR2: 0 = PLL running */
00062  k_oscovfsr_moovf = 0x01U, /**< OSCOVFSR bit 0: MOSC stable */
00063  k_oscovfsr_plovf = 0x04U, /**< OSCOVFSR bit 2: PLL stable */
00064 } clock_byte_constants_t;
00065
00066 typedef enum : uint32_t {
00067  /* Bounded poll counts (NASA Rule 2). We poll the hardware ready flag
00068  * (OSCOVFSR.MOOVF / .PLOVF) so the actual wait is hardware-determined --
00069  * roughly 10 ms physical time regardless of CPU clock. The upper bound
00070  * just prevents an infinite spin if the crystal is missing. ~500k iters
00071  * at LOCO 240 kHz is ~3 s, which is still much shorter than the prior
00072  * fixed 10-second busy-wait. */
00073  k_main_osc_poll_max = 500000U,
00074  k_pll_lock_poll_max = 500000U,
00075 } clock_timeout_t;
00076
00077 /*
=====
00078  * clock_init -- bring the chip from POR defaults to PLL 240 / PCKA 120 MHz.
00079  *
=====
*/
00080 void clock_init(void)
00081 {
00082  volatile rx_system_regs_t *sys = system_regs();
00083
00084  /* Unlock PRC0 (clock) + PRC1 (module stop) write protection. */
00085  *prcr_reg() = k_rx_prcr_unlock_prc0_prc1;
00086
00087  /* Stop sub-clock oscillator -- not used on STAR. */
00088  sys->sosccr = k_sub_clock_stopped;
00089
00090  /* Start MOSC (24 MHz external crystal). */
00091  sys->mosccr = k_main_osc_enabled;
00092
00093  /* Poll the hardware MOSC-stable flag. Exits as soon as the crystal has
00094  * settled (typically ~10 ms physical time) instead of waiting a fixed
00095  * 2.4M cycles, which on the LOCO 240 kHz boot clock is ~10 seconds. */
00096  for (volatile uint32_t i = 0; i < k_main_osc_poll_max; i++) {
00097    if ((sys->oscovfsr & k_oscovfsr_moovf) != 0U) {
00098      break;
00099    }
00100  }
00101
00102  /* Configure and enable PLL: 24 MHz x 10 / 1 = 240 MHz. */
00103  sys->pllcr = k_pllcr_mosc_x10;
00104  sys->pllcr2 = k_pll_enabled;
00105
00106  /* Wait for PLL lock (bounded). If we time out, the crystal is missing
00107  * or not driving -- DO NOT switch SCKCR3 to PLL or the chip will halt
00108  * on a dead clock. Stay on LOCO so the LEDs still respond and the user
00109  * can see we got past clock_init. */
00110  bool pll_locked = false;
00111  for (volatile uint32_t i = 0; i < k_pll_lock_poll_max; i++) {
00112    if ((sys->oscovfsr & k_oscovfsr_plovf) != 0U) {
00113      pll_locked = true;
00114      break;
00115    }
00116  }
00117
00118  if (pll_locked) {
00119    /* MANDATORY: raise flash wait state BEFORE switching to >120 MHz. */
00120    *memwait_reg() = k_memwait_one_wait;
00121
00122    /* Apply system clock dividers. */
00123    sys->sckcr = k_sckcr_value;

```

```

00124
00125 /* Route UCLK from main PLL (UPLLSEL=0). */
00126 *((volatile uint16_t *)k_packcr_addr) = 0x0000U;
00127
00128 /* Switch system clock source to PLL. */
00129 sys->sckcr3 = k_sckcr3_cksel_pll;
00130 }
00131
00132 /* Re-lock PRCR. */
00133 *prcr_reg() = k_rx_prcr_lock;
00134 }

```

4.3 imu_test/linker.ld File Reference

4.4 linker.ld

[Go to the documentation of this file.](#)

```

00001 /*
00002  * gpio_test/linker.ld -- Linker script for GPIO breakout board test.
00003  *
00004  * Same memory map as usb_test: 512 KB internal SRAM, 2 MB internal ROM.
00005  * Includes Renesas-syntax sections (P, B, D, C) needed by ThreadX RXv3 port.
00006  */
00007
00008 MEMORY
00009 {
00010     RAM : ORIGIN = 0x00000004, LENGTH = 0x7FFFC /* 512 KB - 4 bytes */
00011     ROM : ORIGIN = 0xFFE00000, LENGTH = 0x200000 /* 2 MB */
00012 }
00013
00014 SECTIONS
00015 {
00016     /* Code -- explicit ROM base so section VMA is anchored correctly. */
00017     .text 0xFFE00000 : AT(0xFFE00000)
00018     {
00019         *(.text*)
00020         *(P)
00021         *(P_1)
00022         *(.rodata*)
00023         *(C)
00024         *(C_1)
00025         *(C_2)
00026         etext = .;
00027     } > ROM
00028
00029     /*
00030     * Relocatable vector table -- startup.S loads INTB with this address.
00031     * Must be 4-byte aligned; only vectors 27 (SWINT) and 28 (CMT0) are
00032     * used, but allocate 256 entries (1024 bytes) for safety.
00033     */
00034     .rvecors ALIGN(4) :
00035     {
00036         __rvecors_start = .;
00037         KEEP(*(.rvecors))
00038         __rvecors_end = .;
00039     } > ROM
00040
00041     /*
00042     * Initialised data -- LMA stays in ROM, VMA in RAM.
00043     * startup.S copies __data..edata from __mdata on boot.
00044     */
00045     __mdata = .;
00046     .data : AT(__mdata)
00047     {
00048         __data = .;
00049         *(.data*)
00050         *(D)
00051         *(D_1)
00052         *(D_2)
00053         __edata = .;
00054     } > RAM
00055
00056     /* BSS -- zeroed by startup.S */
00057     .bss :
00058     {
00059         __bss = .;
00060         *(.bss*)
00061         *(COMMON)

```

```

00062      *(B)
00063      *(B_1)
00064      *(B_2)
00065      _ebss = .;
00066      . = ALIGN(4);
00067      _end = .;
00068  } > RAM AT>RAM
00069
00070  /*
00071  * Stacks -- USP grows down from top of RAM, ISP 4 KB below that.
00072  */
00073  _ustack = ORIGIN(RAM) + LENGTH(RAM);
00074  _istack = _ustack - 0x1000;
00075
00076  /*
00077  * Exception vector table at fixed hardware address 0xFFFFF80.
00078  * 32 entries -- all unused.
00079  */
00080  .exvectors 0xFFFFF80 : AT(0xFFFFF80)
00081  {
00082      LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF);
00083      LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF);
00084      LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF);
00085      LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF);
00086      LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF);
00087      LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF);
00088      LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF);
00089      LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF);
00090  } > ROM
00091
00092  /* Fixed reset vector at 0xFFFFF80 */
00093  .fvectors 0xFFFFF80 : AT(0xFFFFF80)
00094  {
00095      KEEP(*(.fvectors))
00096  } > ROM
00097  }

```

4.5 imu_test/main.c File Reference

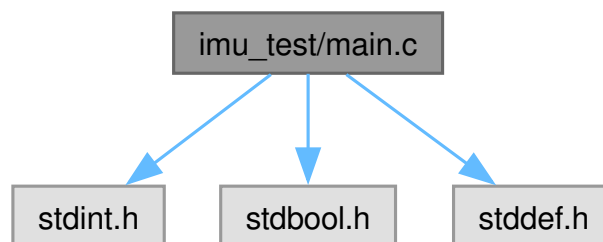
BNO055 + BMP280 over RIIC1 smoke test on the production STAR PCB.

```

#include <stdint.h>
#include <stdbool.h>
#include <stddef.h>

```

Include dependency graph for main.c:



Macros

- #define [REG8\(a\)](#)
- #define [REG16\(a\)](#)
- #define [REG32\(a\)](#)
- #define [I16_LE\(p, i\)](#)

Enumerations

- enum [delay_const_t](#) : uint32_t { [k_iters_per_ms](#) = 40000U }
- enum [gpio_pins_t](#) : uint8_t { [k_port_8](#) = 8 , [k_bit_imu_rst](#) = 3 }
- enum [port_base_t](#) : uintptr_t { [k_pdr_base](#) = 0x0008C000U , [k_podr_base](#) = 0x0008C020U , [k_pmr_base](#) = 0x0008C060U }
- enum [riic_addrs_t](#) : uintptr_t { [k_riic1_base](#) = 0x00088320U , [k_mstpcrb_addr](#) = 0x00080014U , [k_prer_addr](#) = 0x000803FEU , [k_pwpr_addr](#) = 0x0008C11FU }
- enum [riic_off_t](#) : uint8_t { [k_riic_off_iccr1](#) = 0x00 , [k_riic_off_iccr2](#) = 0x01 , [k_riic_off_icmr1](#) = 0x02 , [k_riic_off_icmr3](#) = 0x04 , [k_riic_off_icfer](#) = 0x05 , [k_riic_off_icier](#) = 0x07 , [k_riic_off_icsr1](#) = 0x08 , [k_riic_off_icsr2](#) = 0x09 , [k_riic_off_icbrl](#) = 0x10 , [k_riic_off_icbrh](#) = 0x11 , [k_riic_off_icdrt](#) = 0x12 , [k_riic_off_icdrr](#) = 0x13 }
- enum [riic_bits_t](#) : uint8_t { [k_iccr1_iceen](#) = 0x80 , [k_iccr1_iicrst](#) = 0x40 , [k_iccr2_bbsy](#) = 0x80 , [k_iccr2_mst](#) = 0x40 , [k_iccr2_trs](#) = 0x20 , [k_iccr2_st](#) = 0x02 , [k_iccr2_sp](#) = 0x08 , [k_iccr2_rs](#) = 0x04 , [k_icsr2_al](#) = 0x02 , [k_icsr2_start](#) = 0x04 , [k_icsr2_stop](#) = 0x08 , [k_icsr2_nackf](#) = 0x10 , [k_icsr2_rdrf](#) = 0x20 , [k_icsr2_tend](#) = 0x40 , [k_icsr2_tdre](#) = 0x80 , [k_icmr3_ackbt](#) = 0x08 , [k_icmr3_ackwp](#) = 0x10 , [k_icfer_tmoe](#) = 0x01 , [k_icfer_male](#) = 0x02 , [k_icfer_nale](#) = 0x04 , [k_icfer_sale](#) = 0x08 , [k_icfer_nacke](#) = 0x10 , [k_icfer_nfe](#) = 0x20 , [k_icfer_scle](#) = 0x40 , [k_icfer_fmpe](#) = 0x80 }
- enum [poll_max_t](#) : uint32_t { [k_poll_max](#) = 100000U }
- enum [sensor_t](#) : uint8_t { [k_bno055_addr](#) = 0x28U , [k_bno055_chip_id_reg](#) = 0x00U , [k_bno055_acc_x_lsb](#) = 0x08U , [k_bno055_grv_x_lsb](#) = 0x2EU , [k_bno055_lia_x_lsb](#) = 0x28U , [k_bno055_opr_mode](#) = 0x3DU , [k_bno055_pwr_mode](#) = 0x3EU , [k_bno055_sys_trigger](#) = 0x3FU , [k_bno055_unit_sel](#) = 0x3BU , [k_bno055_page_id](#) = 0x07U , [k_bno055_calib_stat](#) = 0x35U , [k_bno055_mode_config](#) = 0x00U , [k_bno055_mode_ndof](#) = 0x0CU , [k_bno055_pwr_normal](#) = 0x00U , [k_bmp280_addr](#) = 0x76U , [k_bmp280_chip_id_reg](#) = 0xD0U }

Functions

- void [clock_init](#) (void)
- void [sci9_debug_init](#) (void)
- void [sci9_debug_puts](#) (const char *s)
- void [sci9_debug_puthex16](#) (uint16_t v)
- static void [sci9_debug_putint16_col](#) (int16_t v)
- static void [busy_wait_ms](#) (uint32_t ms)
- static void [gpio_output](#) (uint8_t port, uint8_t pin)
- static void [gpio_set](#) (uint8_t port, uint8_t pin, bool high)
- static void [imu_reset_pulse](#) (void)
- static volatile uint8_t * [r1](#) (uint8_t off)
- static void [i2c_bus_recover](#) (void)
- static void [riic1_init](#) (void)
- static void [riic1_recover](#) (void)
- static bool [wait_bit](#) (volatile uint8_t *reg, uint8_t mask, bool want_set)
- static bool [i2c_read_byte](#) (uint8_t dev_addr, uint8_t reg_addr, uint8_t *out)
- static bool [i2c_write_byte](#) (uint8_t dev_addr, uint8_t reg_addr, uint8_t val)
- static bool [i2c_read_bytes](#) (uint8_t dev_addr, uint8_t reg_addr, uint8_t *buf, uint8_t count)
- int [main](#) (void)

Variables

- static volatile uint8_t [s_last_fail_gate](#) = 0U
- static volatile uint8_t [s_last_write_gate](#) = 0U

4.5.1 Detailed Description

BNO055 + BMP280 over RIIC1 smoke test on the production STAR PCB.

Bare-metal direct-register I2C probe – prove the I2C wiring, RIIC1 peripheral config, BNO055 reset+boot, and BMP280 are all healthy. Each step prints over SCI9 BEFORE running so a hang is visible.

Wiring (from src/inc/hardware_config.h): IMU_SCL = P21 / SCL1 pin 36 RIIC1 clock IMU_SDA = P20 / SDA1 pin 37 RIIC1 data IMU_RST = P83 pin 58 active-low BNO055 reset

Devices on RIIC1: BNO055 @ 0x28 – expects chip_id = 0xA0 at reg 0x00 BMP280 @ 0x76 – expects chip_id = 0x58 at reg 0xD0

SPDX-License-Identifier: MIT

Copyright

Copyright (c) 2026 Locked Inc.

Definition in file [main.c](#).

4.5.2 Macro Definition Documentation

4.5.2.1 I16_LE

```
#define I16_LE(
    p,
    i)
```

Value:

```
((int16_t)((uint16_t)(p)[(i)] | ((uint16_t)(p)[(i) + 1U] « 8)))
```

Referenced by [main\(\)](#).

4.5.2.2 REG16

```
#define REG16(
    a)
```

Value:

```
(*(<strong>volatile</strong> uint16_t *)(<strong>uintptr_t</strong>)(a))
```

Definition at line 64 of file [main.c](#).

4.5.2.3 REG32

```
#define REG32(  
    a)
```

Value:

```
(*(volatile uint32_t *) (uintptr_t)(a))
```

Definition at line 65 of file [main.c](#).

4.5.2.4 REG8

```
#define REG8(  
    a)
```

Value:

```
(*(volatile uint8_t *) (uintptr_t)(a))
```

Definition at line 63 of file [main.c](#).

Referenced by [gpio_output\(\)](#), [gpio_set\(\)](#), [i2c_bus_recover\(\)](#), [r1\(\)](#), and [riic1_init\(\)](#).

4.5.3 Enumeration Type Documentation

4.5.3.1 delay_const_t

```
enum delay\_const\_t : uint32_t
```

Enumerator

k_iters_per_ms	
--------------------------------	--

Definition at line 67 of file [main.c](#).

```
00067      : uint32_t {  
00068  k_iters_per_ms = 40000U,  
00069 } delay_const_t;
```

4.5.3.2 gpio_pins_t

```
enum gpio\_pins\_t : uint8_t
```

Enumerator

k_port_8	
k_bit_imu_rst	

Definition at line 83 of file [main.c](#).

```
00083      : uint8_t {  
00084  k_port_8      = 8,  
00085  k_bit_imu_rst = 3, /* P83 = active-low BNO055 reset */  
00086 } gpio_pins_t;
```

4.5.3.3 poll_max_t

```
enum poll_max_t : uint32_t
```

Enumerator

k_poll_max	
------------	--

Definition at line 290 of file [main.c](#).

```
00290      : uint32_t {
00291  k_poll_max = 100000U,          /* ~few ms at -O1 */
00292 } poll_max_t;
```

4.5.3.4 port_base_t

```
enum port_base_t : uintptr_t
```

Enumerator

k_pdr_base	
k_podr_base	
k_pmr_base	

Definition at line 88 of file [main.c](#).

```
00088      : uintptr_t {
00089  k_pdr_base = 0x0008C000U,
00090  k_podr_base = 0x0008C020U,
00091  k_pmr_base = 0x0008C060U,
00092 } port_base_t;
```

4.5.3.5 riic_addrs_t

```
enum riic_addrs_t : uintptr_t
```

Enumerator

k_riic1_base	
k_mstpcrb_addr	
k_prer_addr	
k_pwpr_addr	

Definition at line 120 of file [main.c](#).

```
00120      : uintptr_t {
00121  k_riic1_base = 0x00088320U,
00122  k_mstpcrb_addr = 0x00080014U,
00123  k_prer_addr = 0x000803FEU,
00124  k_pwpr_addr = 0x0008C11FU,
00125 } riic_addrs_t;
```


4.5.3.6 riic_bits_t

```
enum riic_bits_t : uint8_t
```

Enumerator

k_iccr1_iceen	
k_iccr1_iicrst	
k_iccr2_bbsy	
k_iccr2_mst	
k_iccr2_trs	
k_iccr2_st	
k_iccr2_sp	
k_iccr2_rs	
k_icsr2_al	
k_icsr2_start	
k_icsr2_stop	
k_icsr2_nackf	
k_icsr2_rdrf	
k_icsr2_tend	
k_icsr2_tdre	
k_icmr3_ackbt	
k_icmr3_ackwp	
k_icfer_tmoe	
k_icfer_male	
k_icfer_nale	
k_icfer_sale	
k_icfer_nacke	
k_icfer_nfe	
k_icfer_scle	
k_icfer_fmpe	

Definition at line 144 of file [main.c](#).

```

00144      : uint8_t {
00145  k_iccr1_iceen = 0x80,
00146  k_iccr1_iicrst = 0x40,
00147  k_iccr2_bbsy  = 0x80,
00148  k_iccr2_mst   = 0x40, /* Controller mode (writable in config phase) */
00149  k_iccr2_trs   = 0x20, /* Transmit/Receive select (1=transmit) */
00150  k_iccr2_st    = 0x02,
00151  k_iccr2_sp    = 0x08,
00152  k_iccr2_rs    = 0x04,
00153  k_icsr2_al    = 0x02,
00154  k_icsr2_start = 0x04,
00155  k_icsr2_stop  = 0x08,
00156  k_icsr2_nackf = 0x10,
00157  k_icsr2_rdrf  = 0x20,
00158  k_icsr2_tend  = 0x40,
00159  k_icsr2_tdre  = 0x80,
00160  k_icmr3_ackbt = 0x08,
00161  k_icmr3_ackwp = 0x10,
00162  k_icfer_tmoe  = 0x01,
00163  k_icfer_male  = 0x02,
00164  k_icfer_nale  = 0x04,
00165  k_icfer_sale  = 0x08,
00166  k_icfer_nacke = 0x10,

```

```

00167 k_icfer_nfe    = 0x20,
00168 k_icfer_scle   = 0x40,
00169 k_icfer_fmpe  = 0x80,
00170 } riic_bits_t;

```

4.5.3.7 riic_off_t

```
enum riic\_off\_t : uint8_t
```

Enumerator

k_riic_off_iccr1	
k_riic_off_iccr2	
k_riic_off_icmr1	
k_riic_off_icmr3	
k_riic_off_icfer	
k_riic_off_icier	
k_riic_off_icsr1	
k_riic_off_icsr2	
k_riic_off_icbrl	
k_riic_off_icbrh	
k_riic_off_icdrt	
k_riic_off_icdrr	

Definition at line [127](#) of file [main.c](#).

```

00127      : uint8_t {
00128 k_riic_off_iccr1 = 0x00,
00129 k_riic_off_iccr2 = 0x01,
00130 k_riic_off_icmr1 = 0x02,
00131 k_riic_off_icmr3 = 0x04,
00132 k_riic_off_icfer = 0x05,
00133 k_riic_off_icier = 0x07,
00134 k_riic_off_icsr1 = 0x08,
00135 k_riic_off_icsr2 = 0x09,
00136 k_riic_off_icbrl = 0x10,
00137 k_riic_off_icbrh = 0x11,
00138 k_riic_off_icdrt = 0x12,
00139 k_riic_off_icdrr = 0x13,
00140 } riic_off_t;

```

4.5.3.8 sensor_t

```
enum sensor\_t : uint8_t
```

Enumerator

k_bno055_addr	
k_bno055_chip_id_reg	
k_bno055_acc_x_lsb	
k_bno055_grv_x_lsb	
k_bno055_lia_x_lsb	
k_bno055_opr_mode	

k_bno055_pwr_mode	
k_bno055_sys_trigger	
k_bno055_unit_sel	
k_bno055_page_id	
k_bno055_calib_stat	
k_bno055_mode_config	
k_bno055_mode_ndof	
k_bno055_pwr_normal	
k_bmp280_addr	
k_bmp280_chip_id_reg	

Definition at line 514 of file [main.c](#).

```

00514      : uint8_t {
00515  k_bno055_addr      = 0x28U,
00516  k_bno055_chip_id_reg = 0x00U,
00517  k_bno055_acc_x_lsb  = 0x08U,
00518  k_bno055_grv_x_lsb  = 0x2EU, /* Gravity vector base, 6 bytes int16 LE, 1 LSB = 1/100 m/s^2 */
00519  k_bno055_lia_x_lsb  = 0x28U, /* Linear acceleration base (gravity removed by fusion) */
00520  k_bno055_opr_mode    = 0x3DU, /* Operation mode register */
00521  k_bno055_pwr_mode    = 0x3EU, /* Power mode register */
00522  k_bno055_sys_trigger = 0x3FU, /* System trigger (incl. self-test, soft reset) */
00523  k_bno055_unit_sel    = 0x3BU, /* Output units selection */
00524  k_bno055_page_id     = 0x07U, /* Register page selector */
00525  k_bno055_calib_stat  = 0x35U, /* Calibration status (SYS|GYR|ACC|MAG, 2 bits each) */
00526  k_bno055_mode_config = 0x00U, /* OPR_MODE: CONFIGMODE */
00527  k_bno055_mode_ndof   = 0x0CU, /* OPR_MODE: NDOF (full sensor fusion) */
00528  k_bno055_pwr_normal  = 0x00U, /* PWR_MODE: normal */
00529  k_bmp280_addr        = 0x76U,
00530  k_bmp280_chip_id_reg = 0xD0U,
00531 } sensor_t;

```

4.5.4 Function Documentation

4.5.4.1 busy_wait_ms()

```

void busy_wait_ms (
    uint32_t ms) [static]

```

Definition at line 71 of file [main.c](#).

```

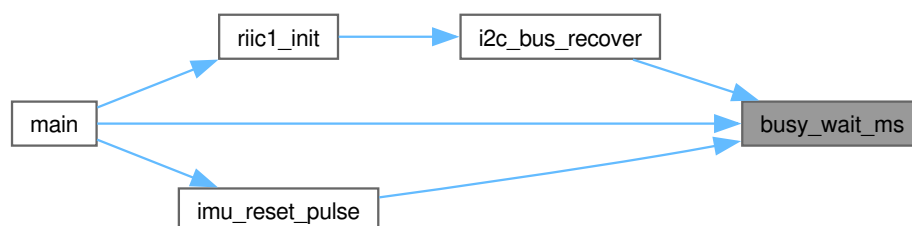
00072 {
00073     for (uint32_t m = 0; m < ms; m++) {
00074         for (volatile uint32_t i = 0; i < k_iters_per_ms; i++) {
00075             __asm__ volatile("nop");
00076         }
00077     }
00078 }

```

References [k_iters_per_ms](#).

Referenced by [i2c_bus_recover\(\)](#), [imu_reset_pulse\(\)](#), and [main\(\)](#).

Here is the caller graph for this function:



4.5.4.2 clock_init()

```
void clock_init (
    void ) [extern]
```

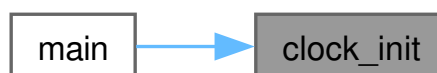
Definition at line 80 of file [clock.c](#).

```
00081 {
00082     volatile rx_system_regs_t *sys = system_regs();
00083
00084     /* Unlock PRC0 (clock) + PRC1 (module stop) write protection. */
00085     *prcr_reg() = k_rx_prcr_unlock_prc0_prc1;
00086
00087     /* Stop sub-clock oscillator -- not used on STAR. */
00088     sys->sosccr = k_sub_clock_stopped;
00089
00090     /* Start MOSC (24 MHz external crystal). */
00091     sys->mosccr = k_main_osc_enabled;
00092
00093     /* Poll the hardware MOSC-stable flag. Exits as soon as the crystal has
00094      * settled (typically ~10 ms physical time) instead of waiting a fixed
00095      * 2.4M cycles, which on the LOCO 240 kHz boot clock is ~10 seconds. */
00096     for (volatile uint32_t i = 0; i < k_main_osc_poll_max; i++) {
00097         if ((sys->oscovfsr & k_oscovfsr_moovf) != 0U) {
00098             break;
00099         }
00100     }
00101
00102     /* Configure and enable PLL: 24 MHz x 10 / 1 = 240 MHz. */
00103     sys->pllcr = k_pllcr_mosc_x10;
00104     sys->pllcr2 = k_pll_enabled;
00105
00106     /* Wait for PLL lock (bounded). If we time out, the crystal is missing
00107      * or not driving -- DO NOT switch SCKCR3 to PLL or the chip will halt
00108      * on a dead clock. Stay on LOCO so the LEDs still respond and the user
00109      * can see we got past clock_init. */
00110     bool pll_locked = false;
00111     for (volatile uint32_t i = 0; i < k_pll_lock_poll_max; i++) {
00112         if ((sys->oscovfsr & k_oscovfsr_plovf) != 0U) {
00113             pll_locked = true;
00114             break;
00115         }
00116     }
00117
00118     if (pll_locked) {
00119         /* MANDATORY: raise flash wait state BEFORE switching to >120 MHz. */
00120         *memwait_reg() = k_memwait_one_wait;
00121
00122         /* Apply system clock dividers. */
00123         sys->sckcr = k_sckcr_value;
00124
00125         /* Route UCLK from main PLL (UPLLSEL=0). */
00126         *((volatile uint16_t *)k_packcr_addr) = 0x0000U;
00127
00128         /* Switch system clock source to PLL. */
00129         sys->sckcr3 = k_sckcr3_cksel_pll;
00130     }
00131
00132     /* Re-lock PRCR. */
00133     *prcr_reg() = k_rx_prcr_lock;
00134 }
```

References [k_main_osc_enabled](#), [k_main_osc_poll_max](#), [k_oscovfsr_moovf](#), [k_oscovfsr_plovf](#), [k_packcr_addr](#), [k_pll_enabled](#), [k_pll_lock_poll_max](#), [k_pllcr_mosc_x10](#), [k_sckcr_value](#), and [k_sub_clock_stopped](#).

Referenced by [main\(\)](#).

Here is the caller graph for this function:



4.5.4.3 gpio_output()

```
void gpio_output (  
    uint8_t port,  
    uint8_t pin)  [inline], [static]
```

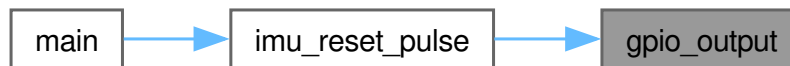
Definition at line 94 of file [main.c](#).

```
00095 {  
00096   REG8(k_pmr_base + port) &= (uint8_t) ~(1U « pin);  
00097   REG8(k_pdr_base + port) |= (uint8_t)(1U « pin);  
00098 }
```

References [k_pdr_base](#), [k_pmr_base](#), and [REG8](#).

Referenced by [imu_reset_pulse\(\)](#).

Here is the caller graph for this function:



4.5.4.4 gpio_set()

```
void gpio_set (  
    uint8_t port,  
    uint8_t pin,  
    bool high)  [inline], [static]
```

Definition at line 99 of file [main.c](#).

```
00100 {  
00101   if (high) { REG8(k_podr_base + port) |= (uint8_t)(1U « pin); }  
00102   else      { REG8(k_podr_base + port) &= (uint8_t) ~(1U « pin); }  
00103 }
```

References [k_podr_base](#), and [REG8](#).

Referenced by [imu_reset_pulse\(\)](#).

Here is the caller graph for this function:



4.5.4.5 i2c_bus_recover()

```
void i2c_bus_recover (
    void ) [static]
```

Definition at line 178 of file [main.c](#).

```
00179 {
00180     REG8(0x0008C062U) &= (uint8_t)~((1U < 0) | (1U < 1)); /* PMR: GPIO mode */
00181     REG8(0x0008C084U) |= (uint8_t)((1U < 1) | (1U < 3)); /* P20/P21 NMOS open-drain */
00182     REG8(0x0008C022U) |= (uint8_t)((1U < 0) | (1U < 1)); /* PODR: idle high (released) */
00183     REG8(0x0008C002U) |= (uint8_t)((1U < 0) | (1U < 1)); /* PDR: outputs */
00184
00185     for (uint8_t i = 0; i < 9U; i++) {
00186         REG8(0x0008C022U) &= (uint8_t)~(1U < 1); /* SCL low */
00187         busy_wait_ms(1);
00188         REG8(0x0008C022U) |= (uint8_t)(1U < 1); /* SCL high */
00189         busy_wait_ms(1);
00190     }
00191     /* Manual STOP: SDA low while SCL high, then SDA high. */
00192     REG8(0x0008C022U) &= (uint8_t)~(1U < 0); /* SDA low */
00193     busy_wait_ms(1);
00194     REG8(0x0008C022U) |= (uint8_t)(1U < 0); /* SDA high (STOP) */
00195     busy_wait_ms(1);
00196
00197     /* Release pins back to input before RIIC1 takes over via PMR=1. RIIC1
00198      * drives the lines on its own once PMR is set; leaving PDR=1 is fine
00199      * per the HW manual but tristating first avoids any glitch. */
00200     REG8(0x0008C002U) &= (uint8_t)~((1U < 0) | (1U < 1));
00201     REG8(0x0008C084U) &= (uint8_t)~((1U < 1) | (1U < 3)); /* clear open-drain */
00202 }
```

References [busy_wait_ms\(\)](#), and [REG8](#).

Referenced by [riic1_init\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.5.4.6 i2c_read_byte()

```
bool i2c_read_byte (
    uint8_t dev_addr,
    uint8_t reg_addr,
    uint8_t * out) [static]
```

Definition at line 309 of file [main.c](#).

```
00310 {
00311     volatile uint8_t *iccr2 = r1(k_riic_off_iccr2);
00312     volatile uint8_t *icsr2 = r1(k_riic_off_icsr2);
00313     volatile uint8_t *icdrt = r1(k_riic_off_icdrt);
00314     volatile uint8_t *icdrr = r1(k_riic_off_icdrr);
00315     volatile uint8_t *icmr3 = r1(k_riic_off_icmr3);
00316
00317     if (!wait_bit(iccr2, k_iccr2_bbsy, false)) {
00318         /* Stuck busy from a previous botched transaction -- nuke the FSM. */
```

```

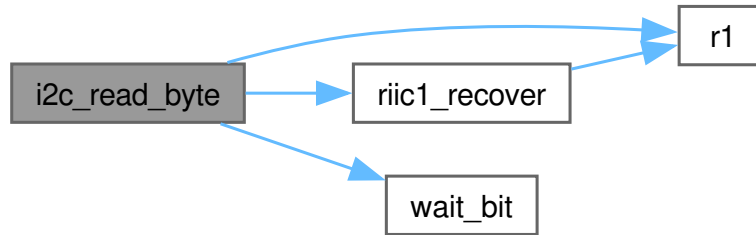
00319     riic1_recover();
00320     if (!wait_bit(icsr2, k_iccr2_bbsy, false)) { return false; }
00321 }
00322 /* Pre-clear any lingering status from the previous read. */
00323 *icsr2 = 0x00U;
00324
00325 /* Controller transmit mode -- mirrors production internal_riic_write_phase().
00326 * Writing MST/TRS while bus is idle is permitted; the bits become read-only
00327 * status during an active transaction. */
00328 *iccr2 = (uint8_t)(k_iccr2_mst | k_iccr2_trs);
00329 *iccr2 |= k_iccr2_st;
00330 if (!wait_bit(icsr2, k_icsr2_start, true)) { s_last_fail_gate = 1U; goto stop_fail; }
00331 *icsr2 &= (uint8_t)~k_icsr2_start;
00332
00333 if (!wait_bit(icsr2, k_icsr2_tdre, true)) { s_last_fail_gate = 2U; goto stop_fail; }
00334 *icdrt = (uint8_t)((dev_addr « 1) | 0U);
00335 if (!wait_bit(icsr2, k_icsr2_tend, true)) { s_last_fail_gate = 3U; goto stop_fail; }
00336 if ((*icsr2) & k_icsr2_nackf) { s_last_fail_gate = 4U; goto stop_fail; }
00337
00338 if (!wait_bit(icsr2, k_icsr2_tdre, true)) { s_last_fail_gate = 5U; goto stop_fail; }
00339 *icdrt = reg_addr;
00340 if (!wait_bit(icsr2, k_icsr2_tend, true)) { s_last_fail_gate = 6U; goto stop_fail; }
00341 if ((*icsr2) & k_icsr2_nackf) { s_last_fail_gate = 7U; goto stop_fail; }
00342
00343 /* Repeated start, then switch to receive mode (TRS=0, MST stays). */
00344 *iccr2 |= k_iccr2_rs;
00345 if (!wait_bit(icsr2, k_icsr2_start, true)) { s_last_fail_gate = 8U; goto stop_fail; }
00346 *icsr2 &= (uint8_t)~k_icsr2_start;
00347 *iccr2 = k_iccr2_mst;
00348
00349 if (!wait_bit(icsr2, k_icsr2_tdre, true)) { s_last_fail_gate = 9U; goto stop_fail; }
00350 *icdrt = (uint8_t)((dev_addr « 1) | 1U);
00351 /* Receive-mode address: skip TEND. After ACK, RIIC immediately clocks the
00352 * first data byte and sets RDRF -- TEND never asserts for the addr|R byte
00353 * in controller-receive mode. NACK still surfaces via NACKF if the
00354 * peripheral didn't answer, so we check that before waiting for RDRF. */
00355 if (!wait_bit(icsr2, (uint8_t)(k_icsr2_rdrf | k_icsr2_nackf), true)) {
00356     s_last_fail_gate = 10U;
00357     goto stop_fail;
00358 }
00359 if ((*icsr2) & k_icsr2_nackf) { s_last_fail_gate = 11U; goto stop_fail; }
00360
00361 *icmr3 |= k_icmr3_ackbt; /* NACK after the byte we want */
00362 (void)*icdrr; /* dummy read kicks reception */
00363
00364 if (!wait_bit(icsr2, k_icsr2_rdrf, true)) { s_last_fail_gate = 12U; goto stop_fail; }
00365
00366 *icsr2 &= (uint8_t)~k_icsr2_stop;
00367 *iccr2 |= k_iccr2_sp;
00368 *out = *icdrr;
00369
00370 if (!wait_bit(icsr2, k_icsr2_stop, true)) { goto cleanup_only; }
00371 cleanup_only:
00372 *icsr2 &= (uint8_t)~(k_icsr2_stop | k_icsr2_nackf);
00373 *icmr3 &= (uint8_t)~k_icmr3_ackbt;
00374 return (*out != 0xFFU || true); /* return true regardless of value */
00375
00376 stop_fail:
00377 *iccr2 |= k_iccr2_sp;
00378 (void)wait_bit(icsr2, k_icsr2_stop, true);
00379 *icsr2 &= (uint8_t)~(k_icsr2_stop | k_icsr2_nackf);
00380 *icmr3 &= (uint8_t)~k_icmr3_ackbt;
00381 return false;
00382 }

```

References [k_iccr2_bbsy](#), [k_iccr2_mst](#), [k_iccr2_rs](#), [k_iccr2_sp](#), [k_iccr2_st](#), [k_iccr2_trs](#), [k_icmr3_ackbt](#), [k_icsr2_nackf](#), [k_icsr2_rdrf](#), [k_icsr2_start](#), [k_icsr2_stop](#), [k_icsr2_tdre](#), [k_icsr2_tend](#), [k_riic_off_iccr2](#), [k_riic_off_icdrr](#), [k_riic_off_icdrt](#), [k_riic_off_icmr3](#), [k_riic_off_icsr2](#), [r1\(\)](#), [riic1_recover\(\)](#), [s_last_fail_gate](#), and [wait_bit\(\)](#).

Referenced by [main\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.5.4.7 i2c_read_bytes()

```

bool i2c_read_bytes (
    uint8_t dev_addr,
    uint8_t reg_addr,
    uint8_t * buf,
    uint8_t count) [static]
  
```

Definition at line 436 of file [main.c](#).

```

00437 {
00438     if (count == 0U) { return false; }
00439
00440     volatile uint8_t *iccr2 = r1(k_riic_off_iccr2);
00441     volatile uint8_t *icsr2 = r1(k_riic_off_icsr2);
00442     volatile uint8_t *icdrt = r1(k_riic_off_icdrt);
00443     volatile uint8_t *icdrr = r1(k_riic_off_icdrr);
00444     volatile uint8_t *icmr3 = r1(k_riic_off_icmr3);
00445
00446     if (!wait_bit(iccr2, k_iccr2_bbsy, false)) {
00447         riic1_recover();
00448         if (!wait_bit(iccr2, k_iccr2_bbsy, false)) { return false; }
00449     }
00450     *icsr2 = 0x00U;
00451
00452     *iccr2 = (uint8_t)(k_iccr2_mst | k_iccr2_trs);
00453     *iccr2 |= k_iccr2_st;
00454     if (!wait_bit(icsr2, k_icsr2_start, true)) { goto rstop_fail; }
00455     *icsr2 &= (uint8_t)~k_icsr2_start;
00456
00457     if (!wait_bit(icsr2, k_icsr2_tdre, true)) { goto rstop_fail; }
00458     *icdrt = (uint8_t)((dev_addr << 1) | 0U);
00459     if (!wait_bit(icsr2, k_icsr2_tend, true)) { goto rstop_fail; }
00460     if ((*icsr2) & k_icsr2_nackf) { goto rstop_fail; }
00461
00462     if (!wait_bit(icsr2, k_icsr2_tdre, true)) { goto rstop_fail; }
00463     *icdrt = reg_addr;
00464     if (!wait_bit(icsr2, k_icsr2_tend, true)) { goto rstop_fail; }
00465     if ((*icsr2) & k_icsr2_nackf) { goto rstop_fail; }
00466
00467     *iccr2 |= k_iccr2_rs;
00468     if (!wait_bit(icsr2, k_icsr2_start, true)) { goto rstop_fail; }
00469     *icsr2 &= (uint8_t)~k_icsr2_start;
00470     *iccr2 = k_iccr2_mst;
00471
00472     if (!wait_bit(icsr2, k_icsr2_tdre, true)) { goto rstop_fail; }
00473     *icdrt = (uint8_t)((dev_addr << 1) | 1U);
00474     if (!wait_bit(icsr2, (uint8_t)(k_icsr2_rdrf | k_icsr2_nackf), true)) { goto rstop_fail; }
00475     if ((*icsr2) & k_icsr2_nackf) { goto rstop_fail; }
00476
00477     /* Dummy read ICDRR to release SCL and start clocking byte 0 into the
00478      * shift register. RX72N HW manual section 38.2.5.3 (controller-receive). */
00479     if (count == 1U) {
00480         /* For a single-byte read, pre-set ACKBT=1 BEFORE the dummy read so the
00481          * byte's ACK slot sends NACK, signalling the peripheral to stop. */
  
```



```

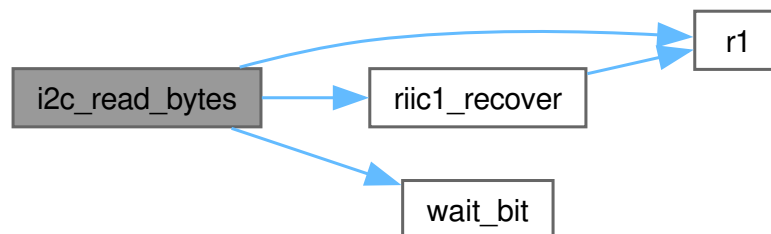
00482  *icmr3 |= k_icmr3_ackbt;
00483  }
00484  (void)*icdrr;
00485
00486  for (uint8_t i = 0; i < count; i++) {
00487      if (!wait_bit(icsr2, k_icsr2_rdrf, true)) { goto rstop_fail; }
00488      if (i == (uint8_t)(count - 2U)) {
00489          /* Next byte is the final one -- set ACKBT=1 so the byte we're about
00490             * to read carries NACK in its 9th-bit slot. */
00491          *icmr3 |= k_icmr3_ackbt;
00492      }
00493      if (i == (uint8_t)(count - 1U)) {
00494          *icsr2 &= (uint8_t)~k_icsr2_stop;
00495          *iccr2 |= k_iccr2_sp;
00496      }
00497      buf[i] = *icdrr;
00498  }
00499
00500  (void)wait_bit(icsr2, k_icsr2_stop, true);
00501  *icsr2 &= (uint8_t)~(k_icsr2_stop | k_icsr2_nackf);
00502  *icmr3 &= (uint8_t)~k_icmr3_ackbt;
00503  return true;
00504
00505 rstop_fail:
00506  *iccr2 |= k_iccr2_sp;
00507  (void)wait_bit(icsr2, k_icsr2_stop, true);
00508  *icsr2 &= (uint8_t)~(k_icsr2_stop | k_icsr2_nackf);
00509  *icmr3 &= (uint8_t)~k_icmr3_ackbt;
00510  return false;
00511 }

```

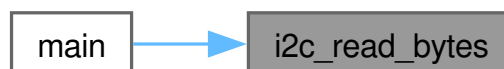
References [k_iccr2_bbsy](#), [k_iccr2_mst](#), [k_iccr2_rs](#), [k_iccr2_sp](#), [k_iccr2_st](#), [k_iccr2_trs](#), [k_icmr3_ackbt](#), [k_icsr2_nackf](#), [k_icsr2_rdrf](#), [k_icsr2_start](#), [k_icsr2_stop](#), [k_icsr2_tdre](#), [k_icsr2_tend](#), [k_riic_off_iccr2](#), [k_riic_off_icdrr](#), [k_riic_off_icdrt](#), [k_riic_off_icmr3](#), [k_riic_off_icsr2](#), [r1\(\)](#), [riic1_recover\(\)](#), and [wait_bit\(\)](#).

Referenced by [main\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.5.4.8 i2c_write_byte()

```

bool i2c_write_byte (
    uint8_t dev_addr,
    uint8_t reg_addr,
    uint8_t val) [static]

```

Definition at line 388 of file [main.c](#).

```

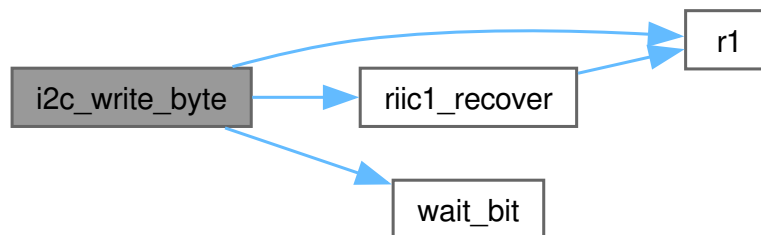
00389 {
00390     volatile uint8_t *iccr2 = r1(k_riic_off_iccr2);
00391     volatile uint8_t *icsr2 = r1(k_riic_off_icsr2);
00392     volatile uint8_t *icdrt = r1(k_riic_off_icdrt);
00393
00394     if (!wait_bit(iccr2, k_iccr2_bbsy, false)) {
00395         riic1_recover();
00396         if (!wait_bit(iccr2, k_iccr2_bbsy, false)) { s_last_write_gate = 1U; return false; }
00397     }
00398     *icsr2 = 0x00U;
00399
00400     *iccr2 = (uint8_t)(k_iccr2_mst | k_iccr2_trs);
00401     *iccr2 |= k_iccr2_st;
00402     if (!wait_bit(icsr2, k_icsr2_start, true)) { s_last_write_gate = 2U; goto wstop_fail; }
00403     *icsr2 &= (uint8_t)~k_icsr2_start;
00404
00405     if (!wait_bit(icsr2, k_icsr2_tdre, true)) { s_last_write_gate = 3U; goto wstop_fail; }
00406     *icdrt = (uint8_t)((dev_addr « 1) | 0U);
00407     if (!wait_bit(icsr2, k_icsr2_tend, true)) { s_last_write_gate = 4U; goto wstop_fail; }
00408     if ((*icsr2) & k_icsr2_nackf) { s_last_write_gate = 5U; goto wstop_fail; }
00409
00410     if (!wait_bit(icsr2, k_icsr2_tdre, true)) { s_last_write_gate = 6U; goto wstop_fail; }
00411     *icdrt = reg_addr;
00412     if (!wait_bit(icsr2, k_icsr2_tend, true)) { s_last_write_gate = 7U; goto wstop_fail; }
00413     if ((*icsr2) & k_icsr2_nackf) { s_last_write_gate = 8U; goto wstop_fail; }
00414
00415     if (!wait_bit(icsr2, k_icsr2_tdre, true)) { s_last_write_gate = 9U; goto wstop_fail; }
00416     *icdrt = val;
00417     if (!wait_bit(icsr2, k_icsr2_tend, true)) { s_last_write_gate = 10U; goto wstop_fail; }
00418     if ((*icsr2) & k_icsr2_nackf) { s_last_write_gate = 11U; goto wstop_fail; }
00419
00420     *icsr2 &= (uint8_t)~k_icsr2_stop;
00421     *iccr2 |= k_iccr2_sp;
00422     (void)wait_bit(icsr2, k_icsr2_stop, true);
00423     *icsr2 &= (uint8_t)~(k_icsr2_stop | k_icsr2_nackf);
00424     return true;
00425
00426 wstop_fail:
00427     *iccr2 |= k_iccr2_sp;
00428     (void)wait_bit(icsr2, k_icsr2_stop, true);
00429     *icsr2 &= (uint8_t)~(k_icsr2_stop | k_icsr2_nackf);
00430     return false;
00431 }

```

References [k_iccr2_bbsy](#), [k_iccr2_mst](#), [k_iccr2_sp](#), [k_iccr2_st](#), [k_iccr2_trs](#), [k_icsr2_nackf](#), [k_icsr2_start](#), [k_icsr2_stop](#), [k_icsr2_tdre](#), [k_icsr2_tend](#), [k_riic_off_iccr2](#), [k_riic_off_icdrt](#), [k_riic_off_icsr2](#), [r1\(\)](#), [riic1_recover\(\)](#), [s_last_write_gate](#), and [wait_bit\(\)](#).

Referenced by [main\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.5.4.9 imu_reset_pulse()

```
void imu_reset_pulse (
    void ) [static]
```

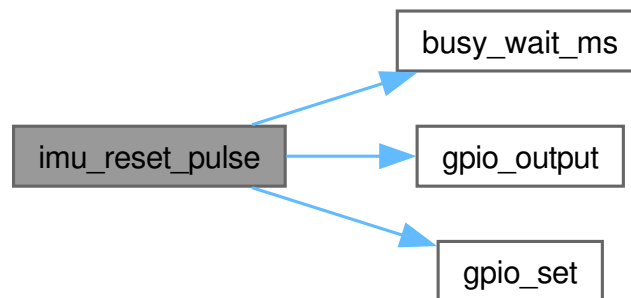
Definition at line 105 of file [main.c](#).

```
00106 {
00107     gpio_output(k_port_8, k_bit_imu_rst);
00108     gpio_set(k_port_8, k_bit_imu_rst, true); /* idle high */
00109     busy_wait_ms(5);
00110     gpio_set(k_port_8, k_bit_imu_rst, false); /* assert reset */
00111     busy_wait_ms(5);
00112     gpio_set(k_port_8, k_bit_imu_rst, true); /* release */
00113     busy_wait_ms(700); /* tBOOT (BNO055 cold start) */
00114 }
```

References [busy_wait_ms\(\)](#), [gpio_output\(\)](#), [gpio_set\(\)](#), [k_bit_imu_rst](#), and [k_port_8](#).

Referenced by [main\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.5.4.10 main()

```
int main (
    void )
```

Definition at line 533 of file [main.c](#).

```
00534 {
00535     clock_init();
00536     sci9_debug_init();
00537     /* 5-second window so the operator can attach `cat /dev/ttyACM0` after a
00538     * fresh flash-and-reset and still see step 1..4 messages. */
00539     busy_wait_ms(5000);
00540     sci9_debug_puts("\n=== IMU_TEST start ===\n");
00541
00542     sci9_debug_puts("step 1: imu_reset_pulse (P83 hi-lo-hi + 700 ms)\n");
00543     imu_reset_pulse();
00544     sci9_debug_puts(" done\n");
00545
00546     sci9_debug_puts("step 2: riic1_init\n");
00547     riic1_init();
00548     sci9_debug_puts(" done\n");
00549 }
```

```

00550 sci9_debug_puts("post-init regs:");
00551 sci9_debug_puts(" ICCR1=0x"); sci9_debug_puthex16((uint16_t)*r1(k_riic_off_iccr1));
00552 sci9_debug_puts(" ICCR2=0x"); sci9_debug_puthex16((uint16_t)*r1(k_riic_off_iccr2));
00553 sci9_debug_puts(" ICSR2=0x"); sci9_debug_puthex16((uint16_t)*r1(k_riic_off_icsr2));
00554 sci9_debug_puts("\n");
00555
00556 sci9_debug_puts("step 3: probe BNO055 chip_id @ 0x28 reg 0x00\n");
00557 uint8_t bno_id = 0xFFU;
00558 bool bno_ok = i2c_read_byte(k_bno055_addr, k_bno055_chip_id_reg, &bno_id);
00559 sci9_debug_puts(bno_ok ? " read ok, chip_id=0x" : " read FAILED, last=0x");
00560 sci9_debug_puthex16((uint16_t)bno_id);
00561 sci9_debug_puts(bno_id == 0xA0U ? " (BNO055 ok)\n" : " (expected 0xA0)\n");
00562
00563 sci9_debug_puts("step 4: probe BMP280 chip_id @ 0x76 reg 0xD0\n");
00564 uint8_t bmp_id = 0xFFU;
00565 bool bmp_ok = i2c_read_byte(k_bmp280_addr, k_bmp280_chip_id_reg, &bmp_id);
00566 sci9_debug_puts(bmp_ok ? " read ok, chip_id=0x" : " read FAILED, last=0x");
00567 sci9_debug_puthex16((uint16_t)bmp_id);
00568 sci9_debug_puts(bmp_id == 0x58U ? " (BMP280 ok)\n" : " (expected 0x58)\n");
00569
00570 sci9_debug_puts("step 5: BNO055 -> NDOF mode (re-init each xact)\n");
00571 riic1_init();
00572 s_last_write_gate = 0U;
00573 bool ok_cfg = i2c_write_byte(k_bno055_addr, k_bno055_opr_mode, k_bno055_mode_config);
00574 uint8_t cfg_gate = s_last_write_gate;
00575 busy_wait_ms(25);
00576 riic1_init();
00577 s_last_write_gate = 0U;
00578 bool ok_ndof = i2c_write_byte(k_bno055_addr, k_bno055_opr_mode, k_bno055_mode_ndof);
00579 uint8_t ndof_gate = s_last_write_gate;
00580 busy_wait_ms(25);
00581 sci9_debug_puts(" cfg="); sci9_debug_puts(ok_cfg ? "ok" : "FAIL");
00582 sci9_debug_puts("@gate=0x"); sci9_debug_puthex16((uint16_t)cfg_gate);
00583 sci9_debug_puts(" ndof="); sci9_debug_puts(ok_ndof ? "ok" : "FAIL");
00584 sci9_debug_puts("@gate=0x"); sci9_debug_puthex16((uint16_t)ndof_gate);
00585 sci9_debug_puts("\n");
00586
00587 sci9_debug_puts("step 6: gravity loop @ 5 Hz (expect |g| ~= 981)\n");
00588 sci9_debug_puts("\n-----\n");
00589 sci9_debug_puts("Watching BNO055 @ 2 Hz. All values are raw int16 from the chip.\n");
00590 sci9_debug_puts("Scale factors (per BNO055 datasheet, units = m/s^2, deg/s, deg, uT):\n");
00591 sci9_debug_puts(" ACC / LIA / GRV: divide by 100 (1 LSB = 0.01 m/s^2)\n");
00592 sci9_debug_puts(" GYR : divide by 16 (1 LSB = 0.0625 deg/s)\n");
00593 sci9_debug_puts(" EUL (hdg/r/p) : divide by 16 (1 LSB = 0.0625 deg)\n");
00594 sci9_debug_puts(" QUA : divide by 16384 (1 LSB = 1 / 2^14)\n");
00595 sci9_debug_puts(" MAG : divide by 16 (1 LSB = 0.0625 uT)\n");
00596 sci9_debug_puts(" temp : deg C (int8 as-is)\n");
00597 sci9_debug_puts(" cal = SYS<6 | GYR<4 | ACC<2 | MAG (each 0.3)\n");
00598 sci9_debug_puts("-----\n");
00599
00600 uint16_t iter = 0;
00601 for (;;) {
00602     /* One burst covers ACC..CALIB_STAT = 46 bytes starting at 0x08. */
00603     uint8_t block[46] = {0};
00604     riic1_init();
00605     bool ok = i2c_read_bytes(k_bno055_addr, k_bno055_acc_x_lsb, block, sizeof(block));
00606
00607     if (!ok) {
00608         sci9_debug_puts("iter=");
00609         sci9_debug_puthex16(iter);
00610         sci9_debug_puts(" READ_FAIL\n");
00611         busy_wait_ms(500);
00612         iter++;
00613         continue;
00614     }
00615
00616     /* Indexing relative to start-register 0x08:
00617     * ACC : 0x08..0x0D -> block[ 0.. 5]
00618     * MAG : 0x0E..0x13 -> block[ 6..11]
00619     * GYR : 0x14..0x19 -> block[12..17]
00620     * EUL : 0x1A..0x1F -> block[18..23] (heading, roll, pitch)
00621     * QUA : 0x20..0x27 -> block[24..31] (w, x, y, z)
00622     * LIA : 0x28..0x2D -> block[32..37]
00623     * GRV : 0x2E..0x33 -> block[38..43]
00624     * TEMP : 0x34 -> block[44] (int8)
00625     * CAL : 0x35 -> block[45] (status byte)
00626     */
00627 #define I16_LE(p, i) \
00628 ((int16_t)((uint16_t)(p)[(i)] | ((uint16_t)(p)[(i) + 1U] << 8)))
00629
00630 int16_t acc_x = I16_LE(block, 0); int16_t acc_y = I16_LE(block, 2); int16_t acc_z = I16_LE(block, 4);
00631 int16_t mag_x = I16_LE(block, 6); int16_t mag_y = I16_LE(block, 8); int16_t mag_z = I16_LE(block, 10);
00632 int16_t gyr_x = I16_LE(block, 12); int16_t gyr_y = I16_LE(block, 14); int16_t gyr_z = I16_LE(block, 16);
00633 int16_t eul_h = I16_LE(block, 18); int16_t eul_r = I16_LE(block, 20); int16_t eul_p = I16_LE(block, 22);
00634 int16_t qua_w = I16_LE(block, 24); int16_t qua_x = I16_LE(block, 26);
00635 int16_t qua_y = I16_LE(block, 28); int16_t qua_z = I16_LE(block, 30);
00636 int16_t lia_x = I16_LE(block, 32); int16_t lia_y = I16_LE(block, 34); int16_t lia_z = I16_LE(block, 36);

```

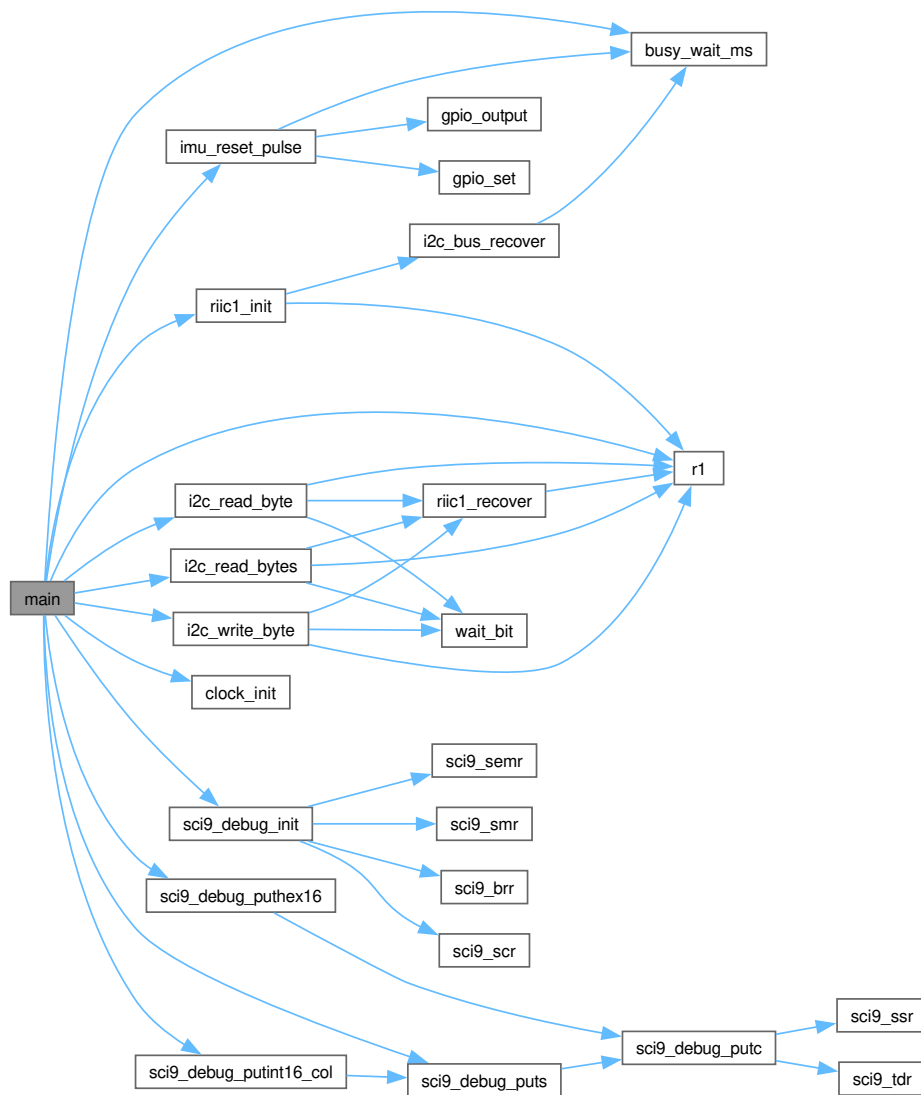
```

00637     int16_t grv_x = I16_LE(block, 38); int16_t grv_y = I16_LE(block, 40); int16_t grv_z = I16_LE(block, 42);
00638     int8_t temp = (int8_t)block[44];
00639     uint8_t calib = block[45];
00640
00641     #undef I16_LE
00642
00643     sci9_debug_puts("iter=");
00644     sci9_debug_puthex16(iter);
00645     sci9_debug_puts("\n");
00646
00647     sci9_debug_puts(" ACC x="); sci9_debug_putint16_col(acc_x);
00648     sci9_debug_puts(" y="); sci9_debug_putint16_col(acc_y);
00649     sci9_debug_puts(" z="); sci9_debug_putint16_col(acc_z);
00650     sci9_debug_puts(" (raw accel, /100 -> m/s^2)\n");
00651
00652     sci9_debug_puts(" MAG x="); sci9_debug_putint16_col(mag_x);
00653     sci9_debug_puts(" y="); sci9_debug_putint16_col(mag_y);
00654     sci9_debug_puts(" z="); sci9_debug_putint16_col(mag_z);
00655     sci9_debug_puts(" (magnetometer, /16 -> uT)\n");
00656
00657     sci9_debug_puts(" GYR x="); sci9_debug_putint16_col(gyr_x);
00658     sci9_debug_puts(" y="); sci9_debug_putint16_col(gyr_y);
00659     sci9_debug_puts(" z="); sci9_debug_putint16_col(gyr_z);
00660     sci9_debug_puts(" (gyroscope, /16 -> deg/s)\n");
00661
00662     sci9_debug_puts(" EUL h="); sci9_debug_putint16_col(eul_h);
00663     sci9_debug_puts(" r="); sci9_debug_putint16_col(eul_r);
00664     sci9_debug_puts(" p="); sci9_debug_putint16_col(eul_p);
00665     sci9_debug_puts(" (euler heading/roll/pitch, /16 -> deg)\n");
00666
00667     sci9_debug_puts(" QUA w="); sci9_debug_putint16_col(qua_w);
00668     sci9_debug_puts(" x="); sci9_debug_putint16_col(qua_x);
00669     sci9_debug_puts(" y="); sci9_debug_putint16_col(qua_y);
00670     sci9_debug_puts(" z="); sci9_debug_putint16_col(qua_z);
00671     sci9_debug_puts(" (quaternion, /16384)\n");
00672
00673     sci9_debug_puts(" LIA x="); sci9_debug_putint16_col(lia_x);
00674     sci9_debug_puts(" y="); sci9_debug_putint16_col(lia_y);
00675     sci9_debug_puts(" z="); sci9_debug_putint16_col(lia_z);
00676     sci9_debug_puts(" (linear accel, gravity removed, /100)\n");
00677
00678     sci9_debug_puts(" GRV x="); sci9_debug_putint16_col(grv_x);
00679     sci9_debug_puts(" y="); sci9_debug_putint16_col(grv_y);
00680     sci9_debug_puts(" z="); sci9_debug_putint16_col(grv_z);
00681     sci9_debug_puts(" (gravity vector, /100)\n");
00682
00683     sci9_debug_puts(" temp="); sci9_debug_putint16_col((int16_t)temp);
00684     sci9_debug_puts(" degC cal=0x"); sci9_debug_puthex16((uint16_t)calib);
00685     sci9_debug_puts(" (SYS="); sci9_debug_putint16_col((int16_t)((calib » 6) & 0x3U));
00686     sci9_debug_puts(" GYR="); sci9_debug_putint16_col((int16_t)((calib » 4) & 0x3U));
00687     sci9_debug_puts(" ACC="); sci9_debug_putint16_col((int16_t)((calib » 2) & 0x3U));
00688     sci9_debug_puts(" MAG="); sci9_debug_putint16_col((int16_t)(calib & 0x3U));
00689     sci9_debug_puts(")\n\n");
00690
00691     busy_wait_ms(500);
00692     iter++;
00693 }
00694 }

```

References [busy_wait_ms\(\)](#), [clock_init\(\)](#), [I16_LE](#), [i2c_read_byte\(\)](#), [i2c_read_bytes\(\)](#), [i2c_write_byte\(\)](#), [imu_reset_pulse\(\)](#), [k_bmp280_addr](#), [k_bmp280_chip_id_reg](#), [k_bno055_acc_x_lsb](#), [k_bno055_addr](#), [k_bno055_chip_id_reg](#), [k_bno055_mode_config](#), [k_bno055_mode_ndof](#), [k_bno055_opr_mode](#), [k_riic_off_iccr1](#), [k_riic_off_iccr2](#), [k_riic_off_icsr2](#), [r1\(\)](#), [riic1_init\(\)](#), [s_last_write_gate](#), [sci9_debug_init\(\)](#), [sci9_debug_puthex16\(\)](#), [sci9_debug_putint16_col\(\)](#), and [sci9_debug_puts\(\)](#).

Here is the call graph for this function:



4.5.4.11 r1()

```
volatile uint8_t * r1 (
    uint8_t off) [inline], [static]
```

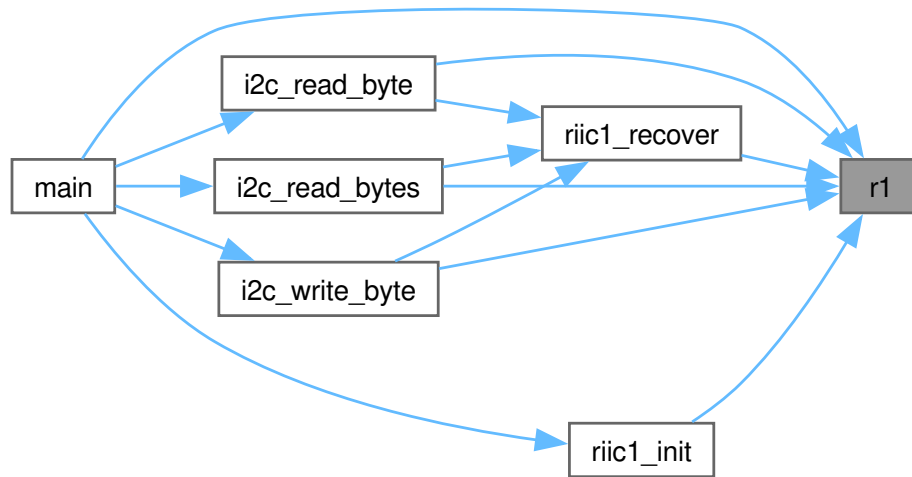
Definition at line 142 of file [main.c](#).

```
00142 { return &REG8(k_riic1_base + off); }
```

References [k_riic1_base](#), and [REG8](#).

Referenced by [i2c_read_byte\(\)](#), [i2c_read_bytes\(\)](#), [i2c_write_byte\(\)](#), [main\(\)](#), [riic1_init\(\)](#), and [riic1_recover\(\)](#).

Here is the caller graph for this function:



4.5.4.12 riic1_init()

```
void riic1_init (
    void ) [static]
```

Definition at line 204 of file [main.c](#).

```

00205 {
00206     /* Step 0: 9-clock SCL bit-bang to flush a stuck peripheral that may be
00207      * holding SDA low from an aborted previous transaction. Runs while pins
00208      * are still GPIO (PMR=0) so we can drive them directly. */
00209     i2c_bus_recover();
00210
00211     /* Step 1: clear MSTPCRB.MSTPB20 (RIIC1) under PRCR unlock 0xA50B
00212      * (PRC1+PRC3; 0xA50F asserts a reserved bit per the cross-check fix). */
00213     *(volatile uint16_t *)k_prcr_addr = 0xA50BU;
00214     *(volatile uint32_t *)k_mstpcrb_addr &= ~(uint32_t)(1UL << 20);
00215     *(volatile uint16_t *)k_prcr_addr = 0xA500U;
00216
00217     /* Step 2: PFS for RIIC1 SDA1=P20, SCL1=P21. PSEL = 0x0F per RX72N
00218      * HW manual chapter 23 Table 23.6. */
00219     *(volatile uint8_t *)k_pwpr_addr = 0x00U;
00220     *(volatile uint8_t *)k_pwpr_addr = 0x40U; /* unlock PFS */
00221     REG8(0x0008C140U + 2U * 8U + 0U) = 0x0FU; /* P20 SDA1 */
00222     REG8(0x0008C140U + 2U * 8U + 1U) = 0x0FU; /* P21 SCL1 */
00223     *(volatile uint8_t *)k_pwpr_addr = 0x00U;
00224     *(volatile uint8_t *)k_pwpr_addr = 0x80U; /* lock PFS */
00225
00226     /* Step 3: PMR=1 to route P20/P21 to RIIC1. */
00227     REG8(0x0008C062U) |= (uint8_t)((1U << 0) | (1U << 1));
00228
00229     /* Step 4: per production riic.c: pulse internal reset (IICRST=1 then
00230      * IICRST=0) so the RIIC enters configuration state with both IICRST
00231      * and ICEEN cleared. Configuration registers are writable in this
00232      * state. Setting ICBRH/ICBRL while IICRST=1 silently no-ops -- we
00233      * already saw that empirically. */
00234     *r1(k_riic_off_iccr1) = k_iccr1_iicrst; /* assert internal reset */
00235     *r1(k_riic_off_iccr1) = 0x00U; /* clear all bits (IICRST=0) */
00236
00237     /* Step 5: bit rate. RX72N HW manual Table 38.x: lower 5 bits hold
00238      * the BRL/BRH count; upper 3 bits are reserved (read-as-1, write
00239      * value ignored). Writing the raw 5-bit value is fine. Production
00240      * uses internal_calculate_bit_rate(); we hard-code values that give
00241      * ~100 kHz at PCLKB=60 MHz (or scaled if PCLKB is actually 120). */
00242     *r1(k_riic_off_icbrh) = 23U;
00243     *r1(k_riic_off_icbtl) = 27U;
00244
00245     /* Step 6: mode/timing. Mirror production riic_init: only program ICMR1
00246      * (controller-mode, 7-bit addressing); leave ICMR2/ICMR3 at reset (0x00)
00247      * and DO NOT touch ICFER -- its reset value 0x72 enables the on-chip
00248      * noise filter (NFE) and clock-sync (SCLE) which the peripheral relies
00249      * on. Overriding it with our own bitmask clobbers NFE and silently
00250      * breaks edge detection. */
00251     *r1(k_riic_off_icmr1) = 0x08U;
00252
00253     /* Step 7: enable peripheral. */
00254     *r1(k_riic_off_iccr1) = k_iccr1_iceen; /* ICEEN=1, IICRST=0 */

```

```

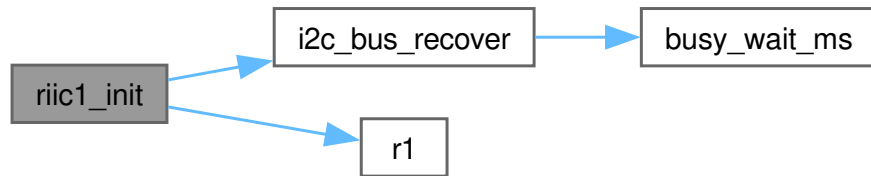
00255
00256 /* Step 8: kick RIIC out of post-enable "stuck low" by clearing SOWP and
00257  * forcing SCLO/SDAO high (open-drain release), then re-locking SOWP. The
00258  * peripheral comes out of IICRST with SCLO=SDAO=0 (driving low) and only
00259  * releases on the first STOP it sees. We synthesize that release by hand. */
00260 uint8_t iccr1 = *r1(k_riic_off_iccr1);
00261 iccr1 &= (uint8_t)~(1U << 4); /* clear SOWP -> writable */
00262 iccr1 |= (uint8_t)((1U << 3) | (1U << 2)); /* SCLO=1, SDAO=1 (release) */
00263 *r1(k_riic_off_iccr1) = iccr1;
00264 iccr1 |= (uint8_t)(1U << 4); /* re-lock SOWP */
00265 *r1(k_riic_off_iccr1) = iccr1;
00266 }

```

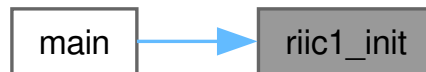
References [i2c_bus_recover\(\)](#), [k_iccr1_iceen](#), [k_iccr1_iicrst](#), [k_mstpcrb_addr](#), [k_prer_addr](#), [k_pwpr_addr](#), [k_riic_off_icbrh](#), [k_riic_off_icbrl](#), [k_riic_off_iccr1](#), [k_riic_off_icmr1](#), [r1\(\)](#), and [REG8](#).

Referenced by [main\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.5.4.13 riic1_recover()

```

void riic1_recover (
    void ) [static]

```

Definition at line 271 of file [main.c](#).

```

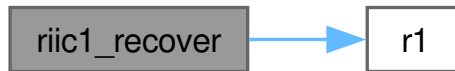
00272 {
00273  *r1(k_riic_off_iccr1) = k_iccr1_iicrst; /* assert IICRST (ICEEN=0) */
00274  *r1(k_riic_off_iccr1) = 0x00U; /* deassert, configuration state */
00275  /* IICRST cycle wipes ICBRH/L/ICMRx/ICFER back to reset (0xFF). Restore. */
00276  *r1(k_riic_off_icbrh) = 23U;
00277  *r1(k_riic_off_icbrl) = 27U;
00278  *r1(k_riic_off_icmr1) = 0x08U;
00279  *r1(k_riic_off_icmr3) = k_icmr3_ackwp;
00280  /* Drop k_icfer_tmoe -- timeout monitoring leaves RIIC stuck if SCL
00281  * doesn't toggle. Rely on our software wait_bit() bound instead. */
00282  *r1(k_riic_off_icfer) = (uint8_t)(k_icfer_male | k_icfer_nale |
00283  k_icfer_sale | k_icfer_nacke | k_icfer_scle);
00284  *r1(k_riic_off_icsr2) = 0x00U; /* clear all status flags */
00285  *r1(k_riic_off_iccr1) = k_iccr1_iceen; /* re-enable */
00286 }

```

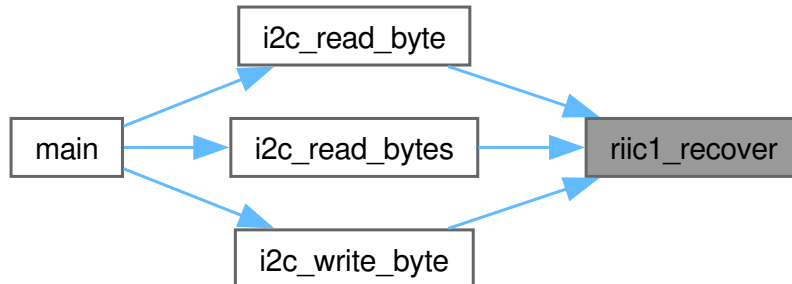
References [k_iccr1_iceen](#), [k_iccr1_iicrst](#), [k_icfer_male](#), [k_icfer_nacke](#), [k_icfer_nale](#), [k_icfer_sale](#), [k_icfer_scle](#), [k_icmr3_ackwp](#), [k_riic_off_icbrh](#), [k_riic_off_icbrl](#), [k_riic_off_iccr1](#), [k_riic_off_icfer](#), [k_riic_off_icmr1](#), [k_riic_off_icmr3](#), [k_riic_off_icsr2](#), and [r1\(\)](#).

Referenced by [i2c_read_byte\(\)](#), [i2c_read_bytes\(\)](#), and [i2c_write_byte\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.5.4.14 sci9_debug_init()

```
void sci9_debug_init (
    void ) [extern]
```

Definition at line 104 of file [sci9.c](#).

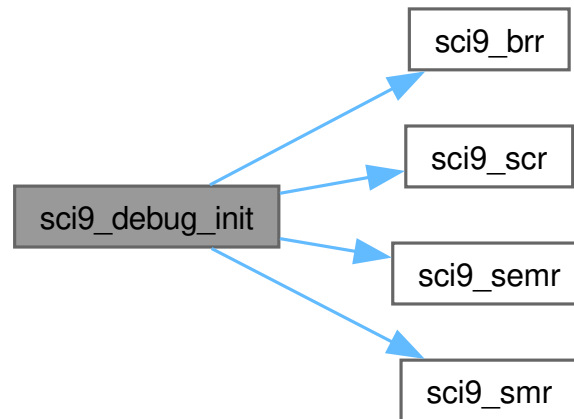
```

00105 {
00106     /* Step 1: release SCI9 from module stop. SCI9 = MSTPCRC bit 26 per
00107      * RX72N HW manual page 410 (NOT MSTPCRB.bit22 -- that's PDC). */
00108     *prcr_reg() = k_rx_prcr_unlock_prc0_prc1;
00109     system_regs()->mstpcrc &= ~(uint32_t)(1UL « (uint32_t)k_mstpc_sci9_bit);
00110     *prcr_reg() = k_rx_prcr_lock;
00111
00112     /* Step 2: configure PB6 (RXD9) and PB7 (TXD9) via MPC. Use the struct
00113      * accessor so the compiler computes the correct PFS offsets via
00114      * static_assert-checked layout. */
00115     mpc()->pwpr = k_pwpr_unlock_b0; /* clear B0WI */
00116     mpc()->pwpr = k_pwpr_pfswe; /* allow PFS writes */
00117     mpc()->pb6pfs = k_psel_sci9;
00118     mpc()->pb7pfs = k_psel_sci9;
00119     mpc()->pwpr = k_pwpr_unlock_b0; /* clear PFSWE */
00120     mpc()->pwpr = k_pwpr_b0wi; /* re-protect */
00121
00122     /* Step 3: PDR before PMR (production uart.c does this). PB7 = output
00123      * for TX, PB6 = input for RX. Then flip PMR to peripheral mode. */
00124     portb()->pdr |= (uint8_t)k_pb7_mask;
00125     portb()->pdr &= (uint8_t)~k_pb6_mask;
00126     portb()->pmr |= (uint8_t)(k_pb6_mask | k_pb7_mask);
00127
00128     /* Step 4: SCI9 init. Match the production order:
00129      * SCR=0 -> SMR -> BRR -> settle -> SCR=TE|RE -> SEMR.
00130      * Don't touch SCMR (reset default 0xF2 is correct). */
00131     *sci9_scr() = k_scr_disabled;
00132     *sci9_smr() = k_smr_8n1_async;
00133     *sci9_brr() = k_brr_115200;
00134
00135     /* Wait roughly one bit time before enabling TX/RX. */
00136     for (volatile uint32_t i = 0U; i < (uint32_t)k_brr_settle_loops; i++) {
00137         __asm__ volatile("nop");
00138     }
00139
00140     *sci9_scr() = k_scr_txrx_enabled;
00141     *sci9_semr() = k_semr_default;
00142 }
```

References [k_brr_115200](#), [k_brr_settle_loops](#), [k_mstpc_sci9_bit](#), [k_pb6_mask](#), [k_pb7_mask](#), [k_psel_sci9](#), [k_pwpr_b0wi](#), [k_pwpr_pfswe](#), [k_pwpr_unlock_b0](#), [k_scr_disabled](#), [k_scr_txrx_enabled](#), [k_semr_default](#), [k_smr_8n1_async](#), [sci9_brr\(\)](#), [sci9_scr\(\)](#), [sci9_semr\(\)](#), and [sci9_smr\(\)](#).

Referenced by [main\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.5.4.15 sci9_debug_puthex16()

```
void sci9_debug_puthex16 (
    uint16_t v) [extern]
```

Definition at line 179 of file [sci9.c](#).

```
00180 {
00181     static const char hex_digits[] = "0123456789ABCDEF";
00182     sci9_debug_putc('0');
00183     sci9_debug_putc('x');
00184     for (int8_t shift = (int8_t)k_hex_shift_init16; shift >= 0;
00185         shift -= (int8_t)k_hex_shift_step) {
00186         sci9_debug_putc(hex_digits[(v » (uint8_t)shift) & (uint8_t)k_hex_nibble_mask]);
00187     }
00188 }
```

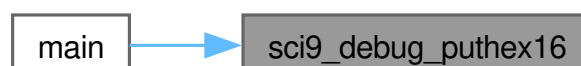
References [k_hex_nibble_mask](#), [k_hex_shift_init16](#), [k_hex_shift_step](#), and [sci9_debug_putc\(\)](#).

Referenced by [main\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.5.4.16 sci9_debug_putint16_col()

```
void sci9_debug_putint16_col (
    int16_t v) [static]
```

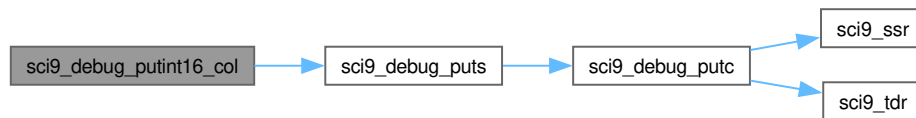
Definition at line 34 of file [main.c](#).

```
00035 {
00036     char    tmp[8] = {0};
00037     uint16_t mag    = (v < 0) ? (uint16_t)-((int32_t)v) : (uint16_t)v;
00038     uint8_t  n      = 0;
00039     if (mag == 0U) {
00040         tmp[n++] = '0';
00041     } else {
00042         while (mag > 0U && n < (uint8_t)(sizeof(tmp) - 1U)) {
00043             tmp[n++] = (char)('0' + (mag % 10U));
00044             mag /= 10U;
00045         }
00046     }
00047     char out[9] = {0};
00048     uint8_t idx = 0;
00049     const uint8_t total = (uint8_t)(n + (v < 0 ? 1U : 0U));
00050     for (uint8_t p = total; p < 7U; p++) {
00051         out[idx++] = ' ';
00052     }
00053     if (v < 0) {
00054         out[idx++] = '-';
00055     }
00056     while (n > 0U) {
00057         out[idx++] = tmp[--n];
00058     }
00059     out[idx] = '\0';
00060     sci9_debug_puts(out);
00061 }
```

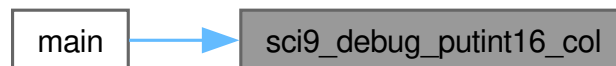
References [sci9_debug_puts\(\)](#).

Referenced by [main\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.5.4.17 sci9_debug_puts()

```
void sci9_debug_puts (
    const char * s) [extern]
```

Definition at line 155 of file [sci9.c](#).

```
00156 {
00157     if (s == (const char *)0) {
00158         return;
00159     }
00160     while (*s != '\0') {
00161         if (*s == '\n') {
00162             sci9_debug_putc('\r');
00163         }
00164         sci9_debug_putc(*s);
00165         s++;
00166     }
00167 }
```

```

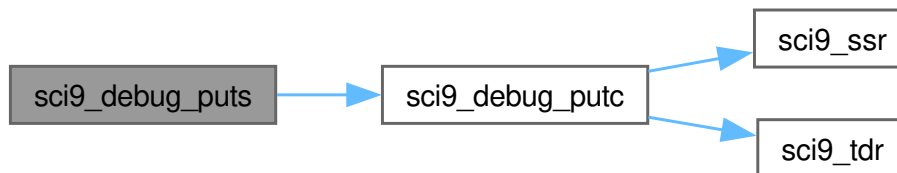
00163     }
00164     sci9_debug_putc(*s++);
00165 }
00166 }

```

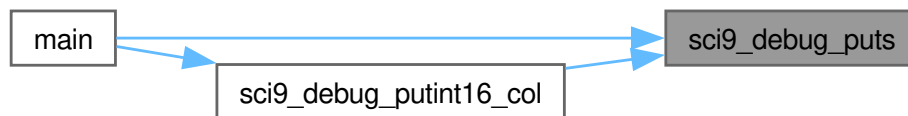
References [sci9_debug_putc\(\)](#).

Referenced by [main\(\)](#), and [sci9_debug_putint16_col\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.5.4.18 wait_bit()

```

bool wait_bit (
    volatile uint8_t * reg,
    uint8_t mask,
    bool want_set) [static]

```

Definition at line 294 of file [main.c](#).

```

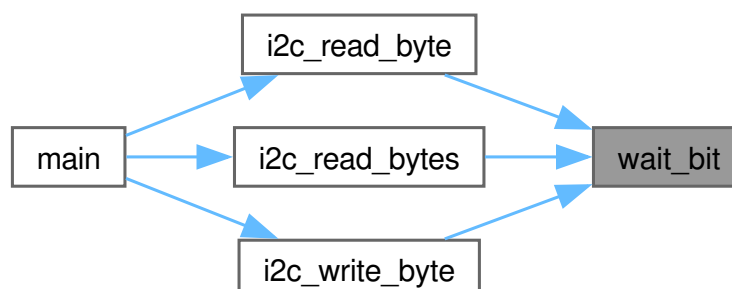
00295 {
00296     for (uint32_t i = 0; i < k_poll_max; i++) {
00297         bool now = ((*reg) & mask) != 0U;
00298         if (now == want_set) { return true; }
00299     }
00300     return false;
00301 }

```

References [k_poll_max](#).

Referenced by [i2c_read_byte\(\)](#), [i2c_read_bytes\(\)](#), and [i2c_write_byte\(\)](#).

Here is the caller graph for this function:



4.5.5 Variable Documentation

4.5.5.1 s_last_fail_gate

volatile uint8_t s_last_fail_gate = 0U [static]

Definition at line 305 of file [main.c](#).

Referenced by [i2c_read_byte\(\)](#).

4.5.5.2 s_last_write_gate

volatile uint8_t s_last_write_gate = 0U [static]

Definition at line 385 of file [main.c](#).

Referenced by [i2c_write_byte\(\)](#), and [main\(\)](#).

4.6 main.c

[Go to the documentation of this file.](#)

```

00001 /**
00002  * @file main.c
00003  * @brief BNO055 + BMP280 over IIC1 smoke test on the production STAR PCB.
00004  *
00005  * Bare-metal direct-register I2C probe -- prove the I2C wiring, IIC1
00006  * peripheral config, BNO055 reset+boot, and BMP280 are all healthy.
00007  * Each step prints over SCI9 BEFORE running so a hang is visible.
00008  *
00009  * Wiring (from src/inc/hardware_config.h):
00010  * IMU_SCL = P21 / SCL1 pin 36 IIC1 clock
00011  * IMU_SDA = P20 / SDA1 pin 37 IIC1 data
00012  * IMU_RST = P83 pin 58 active-low BNO055 reset
00013  *
00014  * Devices on IIC1:
00015  * BNO055 @ 0x28 -- expects chip_id = 0xA0 at reg 0x00
00016  * BMP280 @ 0x76 -- expects chip_id = 0x58 at reg 0xD0
00017  *
00018  * SPDX-License-Identifier: MIT
00019  * @copyright Copyright (c) 2026 Locked Inc.
00020  */
00021
00022 #include <stdint.h>
00023 #include <stdbool.h>
00024 #include <stddef.h>
00025
00026 extern void clock_init(void);
00027 extern void sci9_debug_init(void);
00028 extern void sci9_debug_puts(const char *s);
00029 extern void sci9_debug_puthex16(uint16_t v);
00030
00031 /* Signed-decimal printer for int16 values, right-aligned in 7-char column.
00032  * Builds the string in a local buffer first so sign, digits, and padding
00033  * are emitted in one correct order. */
00034 static void sci9_debug_putint16_col(int16_t v)
00035 {
00036     char tmp[8] = {0};
00037     uint16_t mag = (v < 0) ? (uint16_t)-((int32_t)v) : (uint16_t)v;
00038     uint8_t n = 0;
00039     if (mag == 0U) {
00040         tmp[n++] = '0';
00041     } else {
00042         while (mag > 0U && n < (uint8_t)(sizeof(tmp) - 1U)) {
00043             tmp[n++] = (char)('0' + (mag % 10U));
00044             mag /= 10U;
00045         }
00046     }
00047     char out[9] = {0};
00048     uint8_t idx = 0;

```

```

00049  const uint8_t total = (uint8_t)(n + (v < 0 ? 1U : 0U));
00050  for (uint8_t p = total; p < 7U; p++) {
00051      out[idx++] = ' ';
00052  }
00053  if (v < 0) {
00054      out[idx++] = '-';
00055  }
00056  while (n > 0U) {
00057      out[idx++] = tmp[--n];
00058  }
00059  out[idx] = '\0';
00060  sci9_debug_puts(out);
00061 }
00062
00063 #define REG8(a)  (*(volatile uint8_t *) (uintptr_t)(a))
00064 #define REG16(a) (*(volatile uint16_t *) (uintptr_t)(a))
00065 #define REG32(a) (*(volatile uint32_t *) (uintptr_t)(a))
00066
00067 typedef enum : uint32_t {
00068     k_iters_per_ms = 40000U,
00069 } delay_const_t;
00070
00071 static void busy_wait_ms(uint32_t ms)
00072 {
00073     for (uint32_t m = 0; m < ms; m++) {
00074         for (volatile uint32_t i = 0; i < k_iters_per_ms; i++) {
00075             __asm__ volatile("nop");
00076         }
00077     }
00078 }
00079
00080 /* ----- */
00081 /* GPIO helpers (LEDs + IMU reset on P83) */
00082 /* ----- */
00083 typedef enum : uint8_t {
00084     k_port_8      = 8,
00085     k_bit_imu_rst = 3, /* P83 = active-low BNO055 reset */
00086 } gpio_pins_t;
00087
00088 typedef enum : uintptr_t {
00089     k_pdr_base = 0x0008C000U,
00090     k_podr_base = 0x0008C020U,
00091     k_pmr_base = 0x0008C060U,
00092 } port_base_t;
00093
00094 static inline void gpio_output(uint8_t port, uint8_t pin)
00095 {
00096     REG8(k_pmr_base + port) &= (uint8_t) ~(1U « pin);
00097     REG8(k_pdr_base + port) |= (uint8_t) (1U « pin);
00098 }
00099 static inline void gpio_set(uint8_t port, uint8_t pin, bool high)
00100 {
00101     if (high) { REG8(k_podr_base + port) |= (uint8_t) (1U « pin); }
00102     else      { REG8(k_podr_base + port) &= (uint8_t) ~(1U « pin); }
00103 }
00104
00105 static void imu_reset_pulse(void)
00106 {
00107     gpio_output(k_port_8, k_bit_imu_rst);
00108     gpio_set(k_port_8, k_bit_imu_rst, true); /* idle high */
00109     busy_wait_ms(5);
00110     gpio_set(k_port_8, k_bit_imu_rst, false); /* assert reset */
00111     busy_wait_ms(5);
00112     gpio_set(k_port_8, k_bit_imu_rst, true); /* release */
00113     busy_wait_ms(700); /* tBOOT (BNO055 cold start) */
00114 }
00115
00116 /* ----- */
00117 /* RIIC1 direct-register driver. RIIC1 base = 0x00088321 per RX72N HW */
00118 /* manual chapter 38. Module-stop bit = MSTPCRB.MSTPB20. */
00119 /* ----- */
00120 typedef enum : uintptr_t {
00121     k_riic1_base = 0x00088320U,
00122     k_mstpcrb_addr = 0x00080014U,
00123     k_prcr_addr = 0x000803FEU,
00124     k_pwpr_addr = 0x0008C11FU,
00125 } riic_addrs_t;
00126
00127 typedef enum : uint8_t {
00128     k_riic_off_iccr1 = 0x00,
00129     k_riic_off_iccr2 = 0x01,
00130     k_riic_off_icmr1 = 0x02,
00131     k_riic_off_icmr3 = 0x04,
00132     k_riic_off_icfer = 0x05,
00133     k_riic_off_icier = 0x07,
00134     k_riic_off_icsr1 = 0x08,
00135     k_riic_off_icsr2 = 0x09,

```

```

00136 k_riic_off_icbrl = 0x10,
00137 k_riic_off_icbrh = 0x11,
00138 k_riic_off_icdrt = 0x12,
00139 k_riic_off_icdrr = 0x13,
00140 } riic_off_t;
00141
00142 static inline volatile uint8_t *r1(uint8_t off) { return &REG8(k_riic1_base + off); }
00143
00144 typedef enum : uint8_t {
00145     k_iccr1_iceen = 0x80,
00146     k_iccr1_iicrst = 0x40,
00147     k_iccr2_bbsy = 0x80,
00148     k_iccr2_mst = 0x40, /* Controller mode (writable in config phase) */
00149     k_iccr2_trs = 0x20, /* Transmit/Receive select (1=transmit) */
00150     k_iccr2_st = 0x02,
00151     k_iccr2_sp = 0x08,
00152     k_iccr2_rs = 0x04,
00153     k_icsr2_al = 0x02,
00154     k_icsr2_start = 0x04,
00155     k_icsr2_stop = 0x08,
00156     k_icsr2_nackf = 0x10,
00157     k_icsr2_rdrf = 0x20,
00158     k_icsr2_tend = 0x40,
00159     k_icsr2_tdre = 0x80,
00160     k_icmr3_ackbt = 0x08,
00161     k_icmr3_ackwp = 0x10,
00162     k_icfer_tmoe = 0x01,
00163     k_icfer_male = 0x02,
00164     k_icfer_nale = 0x04,
00165     k_icfer_sale = 0x08,
00166     k_icfer_nacke = 0x10,
00167     k_icfer_nfe = 0x20,
00168     k_icfer_scle = 0x40,
00169     k_icfer_fmpe = 0x80,
00170 } riic_bits_t;
00171
00172 /* Bit-bang SCL 9 times to flush any partial-byte a stuck I2C peripheral may
00173  * be holding SDA low for, then synthesize a STOP. Standard I2C bus-recovery
00174  * dance per NXP UM10204 section 3.1.16. Runs BEFORE we hand the pins to the
00175  * RIIC peripheral. ODR encodes pin drive: 2 bits per pin within ODR0 (pins
00176  * 0-3); the LSB of each pair is reserved, the MSB selects N-ch open-drain.
00177  * For P20: bit 1 of ODR0. For P21: bit 3 of ODR0. */
00178 static void i2c_bus_recover(void)
00179 {
00180     REG8(0x0008C062U) &= (uint8_t)~((1U << 0) | (1U << 1)); /* PMR: GPIO mode */
00181     REG8(0x0008C084U) |= (uint8_t)((1U << 1) | (1U << 3)); /* P20/P21 NMOS open-drain */
00182     REG8(0x0008C022U) |= (uint8_t)((1U << 0) | (1U << 1)); /* PODR: idle high (released) */
00183     REG8(0x0008C002U) |= (uint8_t)((1U << 0) | (1U << 1)); /* PDR: outputs */
00184
00185     for (uint8_t i = 0; i < 9U; i++) {
00186         REG8(0x0008C022U) &= (uint8_t)~(1U << 1); /* SCL low */
00187         busy_wait_ms(1);
00188         REG8(0x0008C022U) |= (uint8_t)(1U << 1); /* SCL high */
00189         busy_wait_ms(1);
00190     }
00191     /* Manual STOP: SDA low while SCL high, then SDA high. */
00192     REG8(0x0008C022U) &= (uint8_t)~(1U << 0); /* SDA low */
00193     busy_wait_ms(1);
00194     REG8(0x0008C022U) |= (uint8_t)(1U << 0); /* SDA high (STOP) */
00195     busy_wait_ms(1);
00196
00197     /* Release pins back to input before RIIC1 takes over via PMR=1. RIIC1
00198     * drives the lines on its own once PMR is set; leaving PDR=1 is fine
00199     * per the HW manual but tristating first avoids any glitch. */
00200     REG8(0x0008C002U) &= (uint8_t)~((1U << 0) | (1U << 1));
00201     REG8(0x0008C084U) &= (uint8_t)~((1U << 1) | (1U << 3)); /* clear open-drain */
00202 }
00203
00204 static void riic1_init(void)
00205 {
00206     /* Step 0: 9-clock SCL bit-bang to flush a stuck peripheral that may be
00207     * holding SDA low from an aborted previous transaction. Runs while pins
00208     * are still GPIO (PMR=0) so we can drive them directly. */
00209     i2c_bus_recover();
00210
00211     /* Step 1: clear MSTPCRB.MSTPB20 (RIIC1) under PRCR unlock 0xA50B
00212     * (PRC1+PRC3; 0xA50F asserts a reserved bit per the cross-check fix). */
00213     *(volatile uint16_t *)k_prcr_addr = 0xA50BU;
00214     *(volatile uint32_t *)k_mstpcrb_addr &= ~(uint32_t)(1UL << 20);
00215     *(volatile uint16_t *)k_prcr_addr = 0xA500U;
00216
00217     /* Step 2: PFS for RIIC1 SDA1=P20, SCL1=P21. PSEL = 0x0F per RX72N
00218     * HW manual chapter 23 Table 23.6. */
00219     *(volatile uint8_t *)k_pwpr_addr = 0x00U;
00220     *(volatile uint8_t *)k_pwpr_addr = 0x40U; /* unlock PFS */
00221     REG8(0x0008C140U + 2U * 8U + 0U) = 0x0FU; /* P20 SDA1 */
00222     REG8(0x0008C140U + 2U * 8U + 1U) = 0x0FU; /* P21 SCL1 */

```

```

00223 *(volatile uint8_t *)k_pwpr_addr = 0x00U;
00224 *(volatile uint8_t *)k_pwpr_addr = 0x80U; /* lock PFS */
00225
00226 /* Step 3: PMR=1 to route P20/P21 to RIIC1. */
00227 REG8(0x0008C062U) |= (uint8_t)((1U << 0) | (1U << 1));
00228
00229 /* Step 4: per production riic.c: pulse internal reset (IICRST=1 then
00230 * IICRST=0) so the RIIC enters configuration state with both IICRST
00231 * and ICEEN cleared. Configuration registers are writable in this
00232 * state. Setting ICBRH/ICBRL while IICRST=1 silently no-ops -- we
00233 * already saw that empirically. */
00234 *r1(k_riic_off_iccr1) = k_iccr1_icrst; /* assert internal reset */
00235 *r1(k_riic_off_iccr1) = 0x00U; /* clear all bits (IICRST=0) */
00236
00237 /* Step 5: bit rate. RX72N HW manual Table 38.x: lower 5 bits hold
00238 * the BRL/BRH count; upper 3 bits are reserved (read-as-1, write
00239 * value ignored). Writing the raw 5-bit value is fine. Production
00240 * uses internal_calculate_bit_rate(); we hard-code values that give
00241 * ~100 kHz at PCLKB=60 MHz (or scaled if PCLKB is actually 120). */
00242 *r1(k_riic_off_icbrh) = 23U;
00243 *r1(k_riic_off_icbrl) = 27U;
00244
00245 /* Step 6: mode/timing. Mirror production riic_init: only program ICMR1
00246 * (controller-mode, 7-bit addressing); leave ICMR2/ICMR3 at reset (0x00)
00247 * and DO NOT touch ICFER -- its reset value 0x72 enables the on-chip
00248 * noise filter (NFE) and clock-sync (SCLE) which the peripheral relies
00249 * on. Overriding it with our own bitmask clobbers NFE and silently
00250 * breaks edge detection. */
00251 *r1(k_riic_off_icmr1) = 0x08U;
00252
00253 /* Step 7: enable peripheral. */
00254 *r1(k_riic_off_iccr1) = k_iccr1_iceen; /* ICEEN=1, IICRST=0 */
00255
00256 /* Step 8: kick RIIC out of post-enable "stuck low" by clearing SOWP and
00257 * forcing SCLO/SDAO high (open-drain release), then re-locking SOWP. The
00258 * peripheral comes out of IICRST with SCLO=SDAO=0 (driving low) and only
00259 * releases on the first STOP it sees. We synthesize that release by hand. */
00260 uint8_t iccr1 = *r1(k_riic_off_iccr1);
00261 iccr1 &= (uint8_t)~(1U << 4); /* clear SOWP -> writable */
00262 iccr1 |= (uint8_t)((1U << 3) | (1U << 2)); /* SCLO=1, SDAO=1 (release) */
00263 *r1(k_riic_off_iccr1) = iccr1;
00264 iccr1 |= (uint8_t)(1U << 4); /* re-lock SOWP */
00265 *r1(k_riic_off_iccr1) = iccr1;
00266 }
00267
00268 /* Force-reset RIIC1 internal state machine (clears stuck BBSY / lingering
00269 * START / NACKF flags from a botched transaction). Cheaper than full
00270 * riic1_init() because we keep PFS / MSTPCR / bit-rate untouched. */
00271 static void riic1_recover(void)
00272 {
00273     *r1(k_riic_off_iccr1) = k_iccr1_icrst; /* assert IICRST (ICEEN=0) */
00274     *r1(k_riic_off_iccr1) = 0x00U; /* deassert, configuration state */
00275     /* IICRST cycle wipes ICBRH/L/ICMRx/ICFER back to reset (0xFF). Restore. */
00276     *r1(k_riic_off_icbrh) = 23U;
00277     *r1(k_riic_off_icbrl) = 27U;
00278     *r1(k_riic_off_icmr1) = 0x08U;
00279     *r1(k_riic_off_icmr3) = k_icmr3_ackwp;
00280     /* Drop k_icfer_tmoe -- timeout monitoring leaves RIIC stuck if SCL
00281     * doesn't toggle. Rely on our software wait_bit() bound instead. */
00282     *r1(k_riic_off_icfer) = (uint8_t)(k_icfer_male | k_icfer_nale |
00283                                     k_icfer_sale | k_icfer_nacke | k_icfer_scle);
00284     *r1(k_riic_off_icsr2) = 0x00U; /* clear all status flags */
00285     *r1(k_riic_off_iccr1) = k_iccr1_iceen; /* re-enable */
00286 }
00287
00288 /* Polling helper with hard upper bound (NASA Rule 2). Returns false on
00289 * timeout instead of hanging. */
00290 typedef enum : uint32_t {
00291     k_poll_max = 10000U, /* ~few ms at -O1 */
00292 } poll_max_t;
00293
00294 static bool wait_bit(volatile uint8_t *reg, uint8_t mask, bool want_set)
00295 {
00296     for (uint32_t i = 0; i < k_poll_max; i++) {
00297         bool now = ((*reg) & mask) != 0U;
00298         if (now == want_set) { return true; }
00299     }
00300     return false;
00301 }
00302
00303 /* Per-gate failure tag, set by i2c_read_byte before a goto stop_fail jump,
00304 * so the caller can print exactly which step bailed out. */
00305 static volatile uint8_t s_last_fail_gate = 0U;
00306
00307 /* Single-byte register read: START -> addr|W -> reg -> RS -> addr|R -> data
00308 * -> NACK -> STOP. Returns false (and forces a STOP) on any NACK/timeout. */
00309 static bool i2c_read_byte(uint8_t dev_addr, uint8_t reg_addr, uint8_t *out)

```



```

00310 {
00311     volatile uint8_t *iccr2 = r1(k_riic_off_iccr2);
00312     volatile uint8_t *icsr2 = r1(k_riic_off_icsr2);
00313     volatile uint8_t *icdrt = r1(k_riic_off_icdrt);
00314     volatile uint8_t *icdrr = r1(k_riic_off_icdrr);
00315     volatile uint8_t *icmr3 = r1(k_riic_off_icmr3);
00316
00317     if (!wait_bit(iccr2, k_iccr2_bbsy, false)) {
00318         /* Stuck busy from a previous botched transaction -- nuke the FSM. */
00319         riic1_recover();
00320         if (!wait_bit(iccr2, k_iccr2_bbsy, false)) { return false; }
00321     }
00322     /* Pre-clear any lingering status from the previous read. */
00323     *icsr2 = 0x00U;
00324
00325     /* Controller transmit mode -- mirrors production internal_riic_write_phase().
00326     * Writing MST/TRS while bus is idle is permitted; the bits become read-only
00327     * status during an active transaction. */
00328     *iccr2 = (uint8_t)(k_iccr2_mst | k_iccr2_trs);
00329     *iccr2 |= k_iccr2_st;
00330     if (!wait_bit(icsr2, k_icsr2_start, true)) { s_last_fail_gate = 1U; goto stop_fail; }
00331     *icsr2 &= (uint8_t)~k_icsr2_start;
00332
00333     if (!wait_bit(icsr2, k_icsr2_tdre, true)) { s_last_fail_gate = 2U; goto stop_fail; }
00334     *icdrt = (uint8_t)((dev_addr « 1) | 0U);
00335     if (!wait_bit(icsr2, k_icsr2_tend, true)) { s_last_fail_gate = 3U; goto stop_fail; }
00336     if ((*icsr2) & k_icsr2_nackf) { s_last_fail_gate = 4U; goto stop_fail; }
00337
00338     if (!wait_bit(icsr2, k_icsr2_tdre, true)) { s_last_fail_gate = 5U; goto stop_fail; }
00339     *icdrt = reg_addr;
00340     if (!wait_bit(icsr2, k_icsr2_tend, true)) { s_last_fail_gate = 6U; goto stop_fail; }
00341     if ((*icsr2) & k_icsr2_nackf) { s_last_fail_gate = 7U; goto stop_fail; }
00342
00343     /* Repeated start, then switch to receive mode (TRS=0, MST stays). */
00344     *iccr2 |= k_iccr2_rs;
00345     if (!wait_bit(icsr2, k_icsr2_start, true)) { s_last_fail_gate = 8U; goto stop_fail; }
00346     *icsr2 &= (uint8_t)~k_icsr2_start;
00347     *iccr2 = k_iccr2_mst;
00348
00349     if (!wait_bit(icsr2, k_icsr2_tdre, true)) { s_last_fail_gate = 9U; goto stop_fail; }
00350     *icdrt = (uint8_t)((dev_addr « 1) | 1U);
00351     /* Receive-mode address: skip TEND. After ACK, RIIC immediately clocks the
00352     * first data byte and sets RDRF -- TEND never asserts for the addr|R byte
00353     * in controller-receive mode. NACK still surfaces via NACKF if the
00354     * peripheral didn't answer, so we check that before waiting for RDRF. */
00355     if (!wait_bit(icsr2, (uint8_t)(k_icsr2_rdrf | k_icsr2_nackf), true)) {
00356         s_last_fail_gate = 10U;
00357         goto stop_fail;
00358     }
00359     if ((*icsr2) & k_icsr2_nackf) { s_last_fail_gate = 11U; goto stop_fail; }
00360
00361     *icmr3 |= k_icmr3_ackbt; /* NACK after the byte we want */
00362     (void)*icdrr; /* dummy read kicks reception */
00363
00364     if (!wait_bit(icsr2, k_icsr2_rdrf, true)) { s_last_fail_gate = 12U; goto stop_fail; }
00365
00366     *icsr2 &= (uint8_t)~k_icsr2_stop;
00367     *iccr2 |= k_iccr2_sp;
00368     *out = *icdrr;
00369
00370     if (!wait_bit(icsr2, k_icsr2_stop, true)) { goto cleanup_only; }
00371 cleanup_only:
00372     *icsr2 &= (uint8_t)~(k_icsr2_stop | k_icsr2_nackf);
00373     *icmr3 &= (uint8_t)~k_icmr3_ackbt;
00374     return (*out != 0xFFU || true); /* return true regardless of value */
00375
00376 stop_fail:
00377     *iccr2 |= k_iccr2_sp;
00378     (void)wait_bit(icsr2, k_icsr2_stop, true);
00379     *icsr2 &= (uint8_t)~(k_icsr2_stop | k_icsr2_nackf);
00380     *icmr3 &= (uint8_t)~k_icmr3_ackbt;
00381     return false;
00382 }
00383
00384 /* Per-write gate trace, populated by i2c_write_byte before goto wstop_fail. */
00385 static volatile uint8_t s_last_write_gate = 0U;
00386
00387 /* Single-byte register write: START -> addr|W -> reg -> val -> STOP. */
00388 static bool i2c_write_byte(uint8_t dev_addr, uint8_t reg_addr, uint8_t val)
00389 {
00390     volatile uint8_t *iccr2 = r1(k_riic_off_iccr2);
00391     volatile uint8_t *icsr2 = r1(k_riic_off_icsr2);
00392     volatile uint8_t *icdrt = r1(k_riic_off_icdrt);
00393
00394     if (!wait_bit(iccr2, k_iccr2_bbsy, false)) {
00395         riic1_recover();
00396         if (!wait_bit(iccr2, k_iccr2_bbsy, false)) { s_last_write_gate = 1U; return false; }

```

```

00397 }
00398 *icr2 = 0x00U;
00399
00400 *iccr2 = (uint8_t)(k_iccr2_mst | k_iccr2_trs);
00401 *iccr2 |= k_iccr2_st;
00402 if (!wait_bit(icsr2, k_icsr2_start, true)) { s_last_write_gate = 2U; goto wstop_fail; }
00403 *icr2 &= (uint8_t)~k_icsr2_start;
00404
00405 if (!wait_bit(icsr2, k_icsr2_tdre, true)) { s_last_write_gate = 3U; goto wstop_fail; }
00406 *icdr2 = (uint8_t)((dev_addr << 1) | 0U);
00407 if (!wait_bit(icsr2, k_icsr2_tend, true)) { s_last_write_gate = 4U; goto wstop_fail; }
00408 if ((*icr2) & k_icsr2_nackf) { s_last_write_gate = 5U; goto wstop_fail; }
00409
00410 if (!wait_bit(icsr2, k_icsr2_tdre, true)) { s_last_write_gate = 6U; goto wstop_fail; }
00411 *icdr2 = reg_addr;
00412 if (!wait_bit(icsr2, k_icsr2_tend, true)) { s_last_write_gate = 7U; goto wstop_fail; }
00413 if ((*icr2) & k_icsr2_nackf) { s_last_write_gate = 8U; goto wstop_fail; }
00414
00415 if (!wait_bit(icsr2, k_icsr2_tdre, true)) { s_last_write_gate = 9U; goto wstop_fail; }
00416 *icdr2 = val;
00417 if (!wait_bit(icsr2, k_icsr2_tend, true)) { s_last_write_gate = 10U; goto wstop_fail; }
00418 if ((*icr2) & k_icsr2_nackf) { s_last_write_gate = 11U; goto wstop_fail; }
00419
00420 *icr2 &= (uint8_t)~k_icsr2_stop;
00421 *iccr2 |= k_iccr2_sp;
00422 (void)wait_bit(icsr2, k_icsr2_stop, true);
00423 *icr2 &= (uint8_t)~(k_icsr2_stop | k_icsr2_nackf);
00424 return true;
00425
00426 wstop_fail:
00427 *iccr2 |= k_iccr2_sp;
00428 (void)wait_bit(icsr2, k_icsr2_stop, true);
00429 *icr2 &= (uint8_t)~(k_icsr2_stop | k_icsr2_nackf);
00430 return false;
00431 }
00432
00433 /* Multi-byte sequential read: START -> addr|W -> reg -> RS -> addr|R ->
00434 * read N bytes (ACK first N-1, NACK last) -> STOP. Used for BNO055 sensor
00435 * burst reads (gravity vector = 6 bytes). */
00436 static bool i2c_read_bytes(uint8_t dev_addr, uint8_t reg_addr, uint8_t *buf, uint8_t count)
00437 {
00438     if (count == 0U) { return false; }
00439
00440     volatile uint8_t *iccr2 = r1(k_riic_off_iccr2);
00441     volatile uint8_t *icr2 = r1(k_riic_off_icsr2);
00442     volatile uint8_t *icdr2 = r1(k_riic_off_icdr2);
00443     volatile uint8_t *icdrr = r1(k_riic_off_icdrr);
00444     volatile uint8_t *icmr3 = r1(k_riic_off_icmr3);
00445
00446     if (!wait_bit(iccr2, k_iccr2_bbsy, false)) {
00447         riic1_recover();
00448         if (!wait_bit(iccr2, k_iccr2_bbsy, false)) { return false; }
00449     }
00450     *icr2 = 0x00U;
00451
00452     *iccr2 = (uint8_t)(k_iccr2_mst | k_iccr2_trs);
00453     *iccr2 |= k_iccr2_st;
00454     if (!wait_bit(icsr2, k_icsr2_start, true)) { goto rstop_fail; }
00455     *icr2 &= (uint8_t)~k_icsr2_start;
00456
00457     if (!wait_bit(icsr2, k_icsr2_tdre, true)) { goto rstop_fail; }
00458     *icdr2 = (uint8_t)((dev_addr << 1) | 0U);
00459     if (!wait_bit(icsr2, k_icsr2_tend, true)) { goto rstop_fail; }
00460     if ((*icr2) & k_icsr2_nackf) { goto rstop_fail; }
00461
00462     if (!wait_bit(icsr2, k_icsr2_tdre, true)) { goto rstop_fail; }
00463     *icdr2 = reg_addr;
00464     if (!wait_bit(icsr2, k_icsr2_tend, true)) { goto rstop_fail; }
00465     if ((*icr2) & k_icsr2_nackf) { goto rstop_fail; }
00466
00467     *iccr2 |= k_iccr2_rs;
00468     if (!wait_bit(icsr2, k_icsr2_start, true)) { goto rstop_fail; }
00469     *icr2 &= (uint8_t)~k_icsr2_start;
00470     *iccr2 = k_iccr2_mst;
00471
00472     if (!wait_bit(icsr2, k_icsr2_tdre, true)) { goto rstop_fail; }
00473     *icdr2 = (uint8_t)((dev_addr << 1) | 1U);
00474     if (!wait_bit(icsr2, (uint8_t)(k_icsr2_rdrf | k_icsr2_nackf), true)) { goto rstop_fail; }
00475     if ((*icr2) & k_icsr2_nackf) { goto rstop_fail; }
00476
00477     /* Dummy read ICDRR to release SCL and start clocking byte 0 into the
00478     * shift register. RX72N HW manual section 38.2.5.3 (controller-receive). */
00479     if (count == 1U) {
00480         /* For a single-byte read, pre-set ACKBT=1 BEFORE the dummy read so the
00481         * byte's ACK slot sends NACK, signalling the peripheral to stop. */
00482         *icmr3 |= k_icmr3_ackbt;
00483     }

```

```

00484 (void)*icdrr;
00485
00486 for (uint8_t i = 0; i < count; i++) {
00487     if (!wait_bit(icsr2, k_icsr2_rdrf, true)) { goto rstop_fail; }
00488     if (i == (uint8_t)(count - 2U)) {
00489         /* Next byte is the final one -- set ACKBT=1 so the byte we're about
00490          * to read carries NACK in its 9th-bit slot. */
00491         *icmr3 |= k_icmr3_ackbt;
00492     }
00493     if (i == (uint8_t)(count - 1U)) {
00494         *icsr2 &= (uint8_t)~k_icsr2_stop;
00495         *iccr2 |= k_iccr2_sp;
00496     }
00497     buf[i] = *icdrr;
00498 }
00499
00500 (void)wait_bit(icsr2, k_icsr2_stop, true);
00501 *icsr2 &= (uint8_t)~(k_icsr2_stop | k_icsr2_nackf);
00502 *icmr3 &= (uint8_t)~k_icmr3_ackbt;
00503 return true;
00504
00505 rstop_fail:
00506 *iccr2 |= k_iccr2_sp;
00507 (void)wait_bit(icsr2, k_icsr2_stop, true);
00508 *icsr2 &= (uint8_t)~(k_icsr2_stop | k_icsr2_nackf);
00509 *icmr3 &= (uint8_t)~k_icmr3_ackbt;
00510 return false;
00511 }
00512
00513 /* ----- */
00514 typedef enum : uint8_t {
00515     k_bno055_addr = 0x28U,
00516     k_bno055_chip_id_reg = 0x00U,
00517     k_bno055_acc_x_lsb = 0x08U,
00518     k_bno055_grv_x_lsb = 0x2EU, /* Gravity vector base, 6 bytes int16 LE, 1 LSB = 1/100 m/s^2 */
00519     k_bno055_lia_x_lsb = 0x28U, /* Linear acceleration base (gravity removed by fusion) */
00520     k_bno055_opr_mode = 0x3DU, /* Operation mode register */
00521     k_bno055_pwr_mode = 0x3EU, /* Power mode register */
00522     k_bno055_sys_trigger = 0x3FU, /* System trigger (incl. self-test, soft reset) */
00523     k_bno055_unit_sel = 0x3BU, /* Output units selection */
00524     k_bno055_page_id = 0x07U, /* Register page selector */
00525     k_bno055_calib_stat = 0x35U, /* Calibration status (SYS|GYR|ACC|MAG, 2 bits each) */
00526     k_bno055_mode_config = 0x00U, /* OPR_MODE: CONFIGMODE */
00527     k_bno055_mode_ndof = 0x0CU, /* OPR_MODE: NDOF (full sensor fusion) */
00528     k_bno055_pwr_normal = 0x00U, /* PWR_MODE: normal */
00529     k_bmp280_addr = 0x76U,
00530     k_bmp280_chip_id_reg = 0xD0U,
00531 } sensor_t;
00532
00533 int main(void)
00534 {
00535     clock_init();
00536     sci9_debug_init();
00537     /* 5-second window so the operator can attach `cat /dev/ttyACM0` after a
00538      * fresh flash-and-reset and still see step 1..4 messages. */
00539     busy_wait_ms(5000);
00540     sci9_debug_puts("\n=== IMU_TEST start ===\n");
00541
00542     sci9_debug_puts("step 1: imu_reset_pulse (P83 hi-lo-hi + 700 ms)\n");
00543     imu_reset_pulse();
00544     sci9_debug_puts(" done\n");
00545
00546     sci9_debug_puts("step 2: riic1_init\n");
00547     riic1_init();
00548     sci9_debug_puts(" done\n");
00549
00550     sci9_debug_puts("post-init regs:");
00551     sci9_debug_puts(" ICCR1=0x"); sci9_debug_puthex16((uint16_t)*r1(k_riic_off_iccr1));
00552     sci9_debug_puts(" ICCR2=0x"); sci9_debug_puthex16((uint16_t)*r1(k_riic_off_iccr2));
00553     sci9_debug_puts(" ICSR2=0x"); sci9_debug_puthex16((uint16_t)*r1(k_riic_off_icsr2));
00554     sci9_debug_puts("\n");
00555
00556     sci9_debug_puts("step 3: probe BNO055 chip_id @ 0x28 reg 0x00\n");
00557     uint8_t bno_id = 0xFFU;
00558     bool bno_ok = i2c_read_byte(k_bno055_addr, k_bno055_chip_id_reg, &bno_id);
00559     sci9_debug_puts(bno_ok ? " read ok, chip_id=0x" : " read FAILED, last=0x");
00560     sci9_debug_puthex16((uint16_t)bno_id);
00561     sci9_debug_puts(bno_id == 0xA0U ? " (BNO055 ok)\n" : " (expected 0xA0)\n");
00562
00563     sci9_debug_puts("step 4: probe BMP280 chip_id @ 0x76 reg 0xD0\n");
00564     uint8_t bmp_id = 0xFFU;
00565     bool bmp_ok = i2c_read_byte(k_bmp280_addr, k_bmp280_chip_id_reg, &bmp_id);
00566     sci9_debug_puts(bmp_ok ? " read ok, chip_id=0x" : " read FAILED, last=0x");
00567     sci9_debug_puthex16((uint16_t)bmp_id);
00568     sci9_debug_puts(bmp_id == 0x58U ? " (BMP280 ok)\n" : " (expected 0x58)\n");
00569
00570     sci9_debug_puts("step 5: BNO055 -> NDOF mode (re-init each xact)\n");

```

```

00571 riic1_init();
00572 s_last_write_gate = 0U;
00573 bool ok_cfg = i2c_write_byte(k_bno055_addr, k_bno055_opr_mode, k_bno055_mode_config);
00574 uint8_t cfg_gate = s_last_write_gate;
00575 busy_wait_ms(25);
00576 riic1_init();
00577 s_last_write_gate = 0U;
00578 bool ok_ndof = i2c_write_byte(k_bno055_addr, k_bno055_opr_mode, k_bno055_mode_ndof);
00579 uint8_t ndof_gate = s_last_write_gate;
00580 busy_wait_ms(25);
00581 sci9_debug_puts("  cfg="); sci9_debug_puts(ok_cfg ? "ok" : "FAIL");
00582 sci9_debug_puts("@gate=0x"); sci9_debug_puthex16((uint16_t)cfg_gate);
00583 sci9_debug_puts("  ndof="); sci9_debug_puts(ok_ndof ? "ok" : "FAIL");
00584 sci9_debug_puts("@gate=0x"); sci9_debug_puthex16((uint16_t)ndof_gate);
00585 sci9_debug_puts("\n");
00586
00587 sci9_debug_puts("step 6: gravity loop @ 5 Hz (expect |g| ~= 981)\n");
00588 sci9_debug_puts("\n-----\n");
00589 sci9_debug_puts("Watching BNO055 @ 2 Hz. All values are raw int16 from the chip.\n");
00590 sci9_debug_puts("Scale factors (per BNO055 datasheet, units = m/s^2, deg/s, deg, uT):\n");
00591 sci9_debug_puts("  ACC / LIA / GRV: divide by 100 (1 LSB = 0.01 m/s^2)\n");
00592 sci9_debug_puts("  GYR      : divide by 16 (1 LSB = 0.0625 deg/s)\n");
00593 sci9_debug_puts("  EUL (hdg/r/p) : divide by 16 (1 LSB = 0.0625 deg)\n");
00594 sci9_debug_puts("  QUA      : divide by 16384 (1 LSB = 1 / 2^14)\n");
00595 sci9_debug_puts("  MAG      : divide by 16 (1 LSB = 0.0625 uT)\n");
00596 sci9_debug_puts("  temp     : deg C (int8 as-is)\n");
00597 sci9_debug_puts("  cal = SYS«6 | GYR«4 | ACC«2 | MAG (each 0.3)\n");
00598 sci9_debug_puts("-----\n");
00599
00600 uint16_t iter = 0;
00601 for (;;) {
00602     /* One burst covers ACC..CALIB_STAT = 46 bytes starting at 0x08. */
00603     uint8_t block[46] = {0};
00604     riic1_init();
00605     bool ok = i2c_read_bytes(k_bno055_addr, k_bno055_acc_x_lsb, block, sizeof(block));
00606
00607     if (lok) {
00608         sci9_debug_puts("iter=");
00609         sci9_debug_puthex16(iter);
00610         sci9_debug_puts("  READ_FAIL\n");
00611         busy_wait_ms(500);
00612         iter++;
00613         continue;
00614     }
00615
00616     /* Indexing relative to start-register 0x08:
00617     *  ACC : 0x08..0x0D -> block[ 0.. 5]
00618     *  MAG : 0x0E..0x13 -> block[ 6..11]
00619     *  GYR : 0x14..0x19 -> block[12..17]
00620     *  EUL : 0x1A..0x1F -> block[18..23] (heading, roll, pitch)
00621     *  QUA : 0x20..0x27 -> block[24..31] (w, x, y, z)
00622     *  LIA : 0x28..0x2D -> block[32..37]
00623     *  GRV : 0x2E..0x33 -> block[38..43]
00624     *  TEMP : 0x34      -> block[44] (int8)
00625     *  CAL : 0x35      -> block[45] (status byte)
00626     */
00627 #define I16_LE(p, i) \
00628 ((int16_t)((uint16_t)(p)[(i)] | ((uint16_t)(p)[(i) + 1U] « 8)))
00629
00630 int16_t acc_x = I16_LE(block, 0); int16_t acc_y = I16_LE(block, 2); int16_t acc_z = I16_LE(block, 4);
00631 int16_t mag_x = I16_LE(block, 6); int16_t mag_y = I16_LE(block, 8); int16_t mag_z = I16_LE(block, 10);
00632 int16_t gyr_x = I16_LE(block, 12); int16_t gyr_y = I16_LE(block, 14); int16_t gyr_z = I16_LE(block, 16);
00633 int16_t eul_h = I16_LE(block, 18); int16_t eul_r = I16_LE(block, 20); int16_t eul_p = I16_LE(block, 22);
00634 int16_t qua_w = I16_LE(block, 24); int16_t qua_x = I16_LE(block, 26);
00635 int16_t qua_y = I16_LE(block, 28); int16_t qua_z = I16_LE(block, 30);
00636 int16_t lia_x = I16_LE(block, 32); int16_t lia_y = I16_LE(block, 34); int16_t lia_z = I16_LE(block, 36);
00637 int16_t grv_x = I16_LE(block, 38); int16_t grv_y = I16_LE(block, 40); int16_t grv_z = I16_LE(block, 42);
00638 int8_t temp = (int8_t)block[44];
00639 uint8_t calib = block[45];
00640
00641 #undef I16_LE
00642
00643 sci9_debug_puts("iter=");
00644 sci9_debug_puthex16(iter);
00645 sci9_debug_puts("\n");
00646
00647 sci9_debug_puts("  ACC x="); sci9_debug_putint16_col(acc_x);
00648 sci9_debug_puts("  y="); sci9_debug_putint16_col(acc_y);
00649 sci9_debug_puts("  z="); sci9_debug_putint16_col(acc_z);
00650 sci9_debug_puts(" (raw accel, /100 -> m/s^2)\n");
00651
00652 sci9_debug_puts("  MAG x="); sci9_debug_putint16_col(mag_x);
00653 sci9_debug_puts("  y="); sci9_debug_putint16_col(mag_y);
00654 sci9_debug_puts("  z="); sci9_debug_putint16_col(mag_z);
00655 sci9_debug_puts(" (magnetometer, /16 -> uT)\n");
00656
00657 sci9_debug_puts("  GYR x="); sci9_debug_putint16_col(gyr_x);

```

```

00658     sci9_debug_puts(" y=");      sci9_debug_putint16_col(gyr_y);
00659     sci9_debug_puts(" z=");      sci9_debug_putint16_col(gyr_z);
00660     sci9_debug_puts(" (gyroscope, /16 -> deg/s)\n");
00661
00662     sci9_debug_puts(" EUL h="); sci9_debug_putint16_col(eul_h);
00663     sci9_debug_puts(" r=");      sci9_debug_putint16_col(eul_r);
00664     sci9_debug_puts(" p=");      sci9_debug_putint16_col(eul_p);
00665     sci9_debug_puts(" (euler heading/roll/pitch, /16 -> deg)\n");
00666
00667     sci9_debug_puts(" QUA w="); sci9_debug_putint16_col(qua_w);
00668     sci9_debug_puts(" x=");      sci9_debug_putint16_col(qua_x);
00669     sci9_debug_puts(" y=");      sci9_debug_putint16_col(qua_y);
00670     sci9_debug_puts(" z=");      sci9_debug_putint16_col(qua_z);
00671     sci9_debug_puts(" (quaternion, /16384)\n");
00672
00673     sci9_debug_puts(" LIA x="); sci9_debug_putint16_col(lia_x);
00674     sci9_debug_puts(" y=");      sci9_debug_putint16_col(lia_y);
00675     sci9_debug_puts(" z=");      sci9_debug_putint16_col(lia_z);
00676     sci9_debug_puts(" (linear accel, gravity removed, /100)\n");
00677
00678     sci9_debug_puts(" GRV x="); sci9_debug_putint16_col(grv_x);
00679     sci9_debug_puts(" y=");      sci9_debug_putint16_col(grv_y);
00680     sci9_debug_puts(" z=");      sci9_debug_putint16_col(grv_z);
00681     sci9_debug_puts(" (gravity vector, /100)\n");
00682
00683     sci9_debug_puts(" temp=");   sci9_debug_putint16_col((int16_t)temp);
00684     sci9_debug_puts(" degC cal=0x"); sci9_debug_puthex16((uint16_t)calib);
00685     sci9_debug_puts(" (SYS=");   sci9_debug_putint16_col((int16_t)((calib » 6) & 0x3U));
00686     sci9_debug_puts(" GYR=");   sci9_debug_putint16_col((int16_t)((calib » 4) & 0x3U));
00687     sci9_debug_puts(" ACC=");   sci9_debug_putint16_col((int16_t)((calib » 2) & 0x3U));
00688     sci9_debug_puts(" MAG=");   sci9_debug_putint16_col((int16_t)(calib & 0x3U));
00689     sci9_debug_puts(")\n\n");
00690
00691     busy_wait_ms(500);
00692     iter++;
00693 }
00694 }

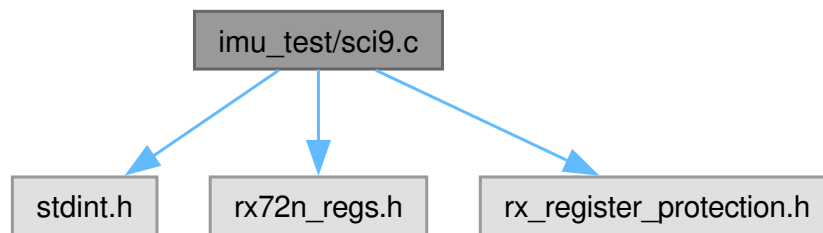
```

4.7 imu_test/sci9.c File Reference

```

#include <stdint.h>
#include "rx72n_regs.h"
#include "rx_register_protection.h"
Include dependency graph for sci9.c:

```



Enumerations

- enum `sci9_addrs_t` : `uintptr_t` { `k_sci9_base` = 0x000D0020U }
- enum `sci9_cfg_t` : `uint8_t` {
`k_psel_sci9` = 0x0AU , `k_pb6_mask` = (uint8_t)(1U << 6U) , `k_pb7_mask` = (uint8_t)(1U << 7U) , `k_mstpc_sci9_bit` = 26U ,
`k_pwpr_unlock_b0` = 0x00U , `k_pwpr_pfswe` = 0x40U , `k_pwpr_b0wi` = 0x80U , `k_brr_115200` = 31U ,
`k_smr_8n1_async` = 0x00U , `k_scr_te_only` = 0x20U , `k_scr_txrx_enabled` = 0x30U ,
`k_scr_disabled` = 0x00U ,
`k_ssr_tdre` = 0x80U , `k_scmr_default` = 0xF2U , `k_semr_default` = 0x00U , `k_hex_nibble_mask` = 0x0FU }
- enum `sci9_settle_t` : `uint16_t` { `k_brr_settle_loops` = 1000U }
- enum `hex_shifts_t` : `uint8_t` { `k_hex_shift_init32` = 28U , `k_hex_shift_init16` = 12U , `k_hex_shift_step` = 4U }

Functions

- static volatile uint8_t * [sci9_smr](#) (void)
- static volatile uint8_t * [sci9_brr](#) (void)
- static volatile uint8_t * [sci9_scr](#) (void)
- static volatile uint8_t * [sci9_tdr](#) (void)
- static volatile uint8_t * [sci9_ssr](#) (void)
- static volatile uint8_t * [sci9_scmr](#) (void)
- static volatile uint8_t * [sci9_semr](#) (void)
- void [sci9_debug_init](#) (void)
- void [sci9_debug_putc](#) (char c)
- void [sci9_debug_puts](#) (const char *s)
- void [sci9_debug_puthex32](#) (uint32_t v)
- void [sci9_debug_puthex16](#) (uint16_t v)

4.7.1 Enumeration Type Documentation

4.7.1.1 hex_shifts_t

enum [hex_shifts_t](#) : uint8_t

Enumerator

k_hex_shift_init32	
k_hex_shift_init16	
k_hex_shift_step	

Definition at line 62 of file [sci9.c](#).

```
00062      : uint8_t {
00063      k_hex_shift_init32 = 28U,
00064      k_hex_shift_init16 = 12U,
00065      k_hex_shift_step  = 4U,
00066 } hex_shifts_t;
```

4.7.1.2 sci9_addrs_t

enum [sci9_addrs_t](#) : uintptr_t

Enumerator

k_sci9_base	
-------------	--

Definition at line 35 of file [sci9.c](#).

```
00035      : uintptr_t {
00036      k_sci9_base = 0x000D0020U,
00037 } sci9_addrs_t;
```

4.7.1.3 sci9_cfg_t

enum [sci9_cfg_t](#) : uint8_t

Enumerator

k_psel_sci9	SCI9 RXD/TXD function
k_pb6_mask	
k_pb7_mask	
k_mstpc_sci9_bit	
k_pwpr_unlock_b0	
k_pwpr_pfswe	
k_pwpr_b0wi	
k_brr_115200	
k_smr_8n1_async	
k_scr_te_only	CKE=00, MPIE=0, RIE=0, TE=1, RE=0
k_scr_txrx_enabled	CKE=00, MPIE=0, RIE=0, TE=1, RE=1
k_scr_disabled	
k_ssr_tdre	
k_scmr_default	
k_semr_default	
k_hex_nibble_mask	

Definition at line 39 of file [sci9.c](#).

```

00039      : uint8_t {
00040      k_psel_sci9      = 0x0AU, /**< SCI9 RXD/TXD function */
00041      k_pb6_mask       = (uint8_t)(1U « 6U),
00042      k_pb7_mask       = (uint8_t)(1U « 7U),
00043      k_mstpc_sci9_bit = 26U, /** MSTPCRC.MSTPC26 per RX72N HW manual page 410 */
00044      k_pwpr_unlock_b0 = 0x00U,
00045      k_pwpr_pfswe     = 0x40U,
00046      k_pwpr_b0wi      = 0x80U,
00047      k_brr_115200     = 31U, /** SCI9 uses PCLKA=120 MHz (SCI7-11 group); BRR=31 @ CKS=0 ABCS=0 -> ~117
                                kbaud (1.7% over 115200) */
00048      k_smr_8n1_async  = 0x00U,
00049      k_scr_te_only    = 0x20U, /**< CKE=00, MPIE=0, RIE=0, TE=1, RE=0 */
00050      k_scr_txrx_enabled = 0x30U, /**< CKE=00, MPIE=0, RIE=0, TE=1, RE=1 */
00051      k_scr_disabled   = 0x00U,
00052      k_ssr_tdre       = 0x80U,
00053      k_scmr_default   = 0xF2U,
00054      k_semr_default   = 0x00U,
00055      k_hex_nibble_mask = 0x0FU,
00056 } sci9_cfg_t;

```

4.7.1.4 sci9_settle_t

enum [sci9_settle_t](#) : uint16_t

Enumerator

k_brr_settle_loops	Match production uart.c (~8.68 us @ 115200)
--------------------	---

Definition at line 58 of file [sci9.c](#).

```

00058      : uint16_t {
00059      k_brr_settle_loops = 1000U, /**< Match production uart.c (~8.68 us @ 115200) */
00060 } sci9_settle_t;

```

4.7.2 Function Documentation

4.7.2.1 sci9_brr()

```
volatile uint8_t * sci9_brr (
    void )    [inline], [static]
```

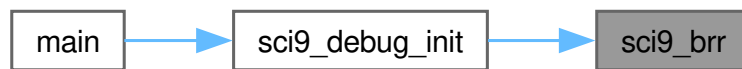
Definition at line 76 of file [sci9.c](#).

```
00077 {
00078     return (volatile uint8_t *) (k_sci9_base + 1U);
00079 }
```

References [k_sci9_base](#).

Referenced by [sci9_debug_init\(\)](#).

Here is the caller graph for this function:



4.7.2.2 sci9_debug_init()

```
void sci9_debug_init (
    void )
```

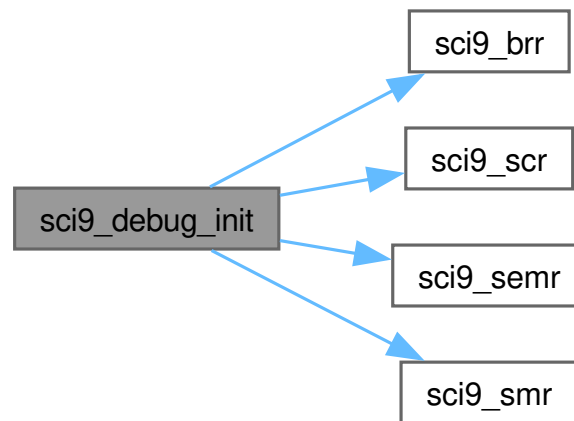
Definition at line 104 of file [sci9.c](#).

```
00105 {
00106     /* Step 1: release SCI9 from module stop. SCI9 = MSTPCRC bit 26 per
00107      * RX72N HW manual page 410 (NOT MSTPCRB.bit22 -- that's PDC). */
00108     *prcr_reg() = k_rx_prcr_unlock_prc0_prc1;
00109     system_regs()->mstpcrc &= ~(uint32_t)(1UL « (uint32_t)k_mstpc_sci9_bit);
00110     *prcr_reg() = k_rx_prcr_lock;
00111
00112     /* Step 2: configure PB6 (RXD9) and PB7 (TXD9) via MPC. Use the struct
00113      * accessor so the compiler computes the correct PFS offsets via
00114      * static_assert-checked layout. */
00115     mpc()->pwpr = k_pwpr_unlock_b0; /* clear B0WI */
00116     mpc()->pwpr = k_pwpr_pfswe; /* allow PFS writes */
00117     mpc()->pb6pfs = k_psel_sci9;
00118     mpc()->pb7pfs = k_psel_sci9;
00119     mpc()->pwpr = k_pwpr_unlock_b0; /* clear PFSWE */
00120     mpc()->pwpr = k_pwpr_b0wi; /* re-protect */
00121
00122     /* Step 3: PDR before PMR (production uart.c does this). PB7 = output
00123      * for TX, PB6 = input for RX. Then flip PMR to peripheral mode. */
00124     portb()->pdr |= (uint8_t)k_pb7_mask;
00125     portb()->pdr &= (uint8_t)~k_pb6_mask;
00126     portb()->pmr |= (uint8_t)(k_pb6_mask | k_pb7_mask);
00127
00128     /* Step 4: SCI9 init. Match the production order:
00129      * SCR=0 -> SMR -> BRR -> settle -> SCR=TE|RE -> SEMR.
00130      * Don't touch SCMR (reset default 0xF2 is correct). */
00131     *sci9_scr() = k_scr_disabled;
00132     *sci9_smr() = k_smr_8n1_async;
00133     *sci9_brr() = k_brr_115200;
00134
00135     /* Wait roughly one bit time before enabling TX/RX. */
00136     for (volatile uint32_t i = 0U; i < (uint32_t)k_brr_settle_loops; i++) {
00137         __asm__ volatile("nop");
00138     }
00139
00140     *sci9_scr() = k_scr_txrx_enabled;
00141     *sci9_semr() = k_semr_default;
00142 }
```

References [k_brr_115200](#), [k_brr_settle_loops](#), [k_mstpc_sci9_bit](#), [k_pb6_mask](#), [k_pb7_mask](#), [k_psel_sci9](#), [k_pwpr_b0wi](#), [k_pwpr_pfswe](#), [k_pwpr_unlock_b0](#), [k_scr_disabled](#), [k_scr_txrx_enabled](#), [k_semr_default](#), [k_smr_8n1_async](#), [sci9_brr\(\)](#), [sci9_scr\(\)](#), [sci9_semr\(\)](#), and [sci9_smr\(\)](#).

Referenced by [main\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.7.2.3 sci9_debug_putc()

```
void sci9_debug_putc (
    char c)
```

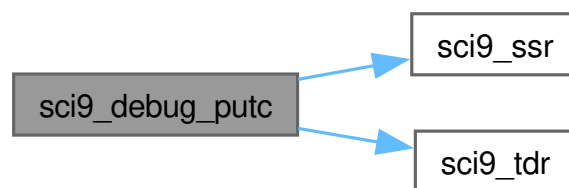
Definition at line 147 of file [sci9.c](#).

```
00148 {
00149     while ((*sci9_ssr() & (uint8_t)k_ssr_tdre) == 0U) {
00150         /* spin */
00151     }
00152     *sci9_tdr() = (uint8_t)c;
00153 }
```

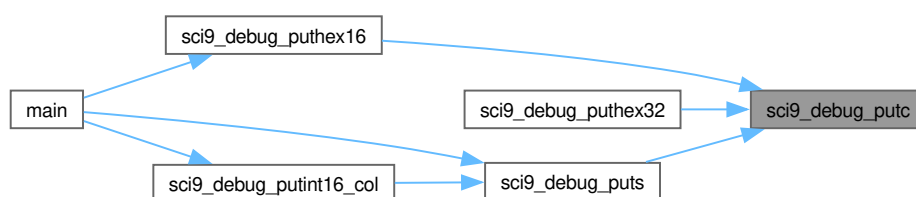
References [k_ssr_tdre](#), [sci9_ssr\(\)](#), and [sci9_tdr\(\)](#).

Referenced by [sci9_debug_puthex16\(\)](#), [sci9_debug_puthex32\(\)](#), and [sci9_debug_puts\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.7.2.4 sci9_debug_puthex16()

```
void sci9_debug_puthex16 (
    uint16_t v)
```

Definition at line 179 of file [sci9.c](#).

```
00180 {
00181     static const char hex_digits[] = "0123456789ABCDEF";
00182     sci9_debug_putc('0');
00183     sci9_debug_putc('x');
00184     for (int8_t shift = (int8_t)k_hex_shift_init16; shift >= 0;
00185          shift -= (int8_t)k_hex_shift_step) {
00186         sci9_debug_putc(hex_digits[(v » (uint8_t)shift) & (uint8_t)k_hex_nibble_mask]);
00187     }
00188 }
```

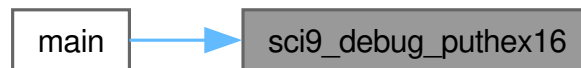
References [k_hex_nibble_mask](#), [k_hex_shift_init16](#), [k_hex_shift_step](#), and [sci9_debug_putc\(\)](#).

Referenced by [main\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.7.2.5 sci9_debug_puthex32()

```
void sci9_debug_puthex32 (
    uint32_t v)
```

Definition at line 168 of file [sci9.c](#).

```
00169 {
00170     static const char hex_digits[] = "0123456789ABCDEF";
00171     sci9_debug_putc('0');
00172     sci9_debug_putc('x');
00173     for (int8_t shift = (int8_t)k_hex_shift_init32; shift >= 0;
00174          shift -= (int8_t)k_hex_shift_step) {
00175         sci9_debug_putc(hex_digits[(v » (uint8_t)shift) & (uint8_t)k_hex_nibble_mask]);
00176     }
00177 }
```

References [k_hex_nibble_mask](#), [k_hex_shift_init32](#), [k_hex_shift_step](#), and [sci9_debug_putc\(\)](#).

Here is the call graph for this function:



4.7.2.6 sci9_debug_puts()

```
void sci9_debug_puts (
    const char * s)
```

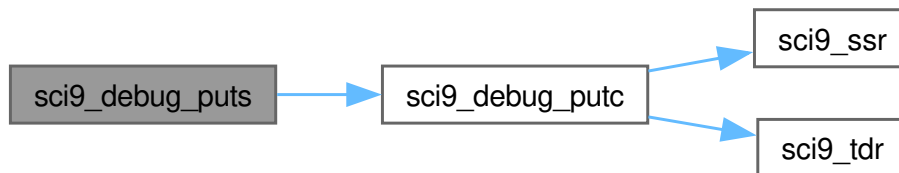
Definition at line 155 of file [sci9.c](#).

```
00156 {
00157     if (s == (const char *)0) {
00158         return;
00159     }
00160     while (*s != '\0') {
00161         if (*s == '\n') {
00162             sci9_debug_putc('\r');
00163         }
00164         sci9_debug_putc(*s++);
00165     }
00166 }
```

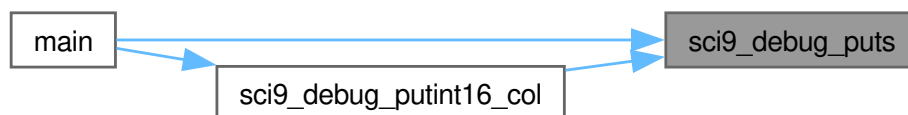
References [sci9_debug_putc\(\)](#).

Referenced by [main\(\)](#), and [sci9_debug_putint16_col\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.7.2.7 sci9_scmr()

```
volatile uint8_t * sci9_scmr (
    void ) [inline], [static]
```

Definition at line 92 of file [sci9.c](#).

```
00093 {
00094     return (volatile uint8_t *) (k_sci9_base + 6U);
00095 }
```

References [k_sci9_base](#).

4.7.2.8 `sci9_scr()`

```
volatile uint8_t * sci9_scr (
    void ) [inline], [static]
```

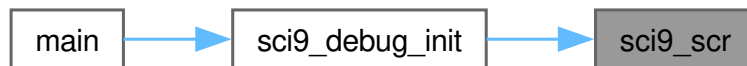
Definition at line 80 of file [sci9.c](#).

```
00081 {
00082     return (volatile uint8_t *) (k_sci9_base + 2U);
00083 }
```

References [k_sci9_base](#).

Referenced by [sci9_debug_init\(\)](#).

Here is the caller graph for this function:

4.7.2.9 `sci9_semr()`

```
volatile uint8_t * sci9_semr (
    void ) [inline], [static]
```

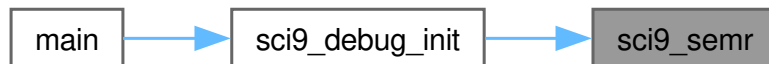
Definition at line 96 of file [sci9.c](#).

```
00097 {
00098     return (volatile uint8_t *) (k_sci9_base + 7U);
00099 }
```

References [k_sci9_base](#).

Referenced by [sci9_debug_init\(\)](#).

Here is the caller graph for this function:

4.7.2.10 `sci9_smr()`

```
volatile uint8_t * sci9_smr (
    void ) [inline], [static]
```

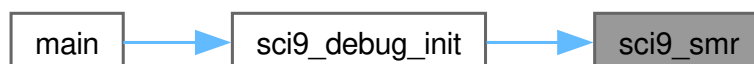
Definition at line 72 of file [sci9.c](#).

```
00073 {
00074     return (volatile uint8_t *) (k_sci9_base + 0U);
00075 }
```

References [k_sci9_base](#).

Referenced by [sci9_debug_init\(\)](#).

Here is the caller graph for this function:



4.7.2.11 sci9_ssr()

```
volatile uint8_t * sci9_ssr (
    void ) [inline], [static]
```

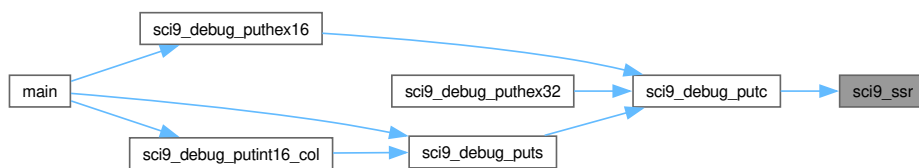
Definition at line 88 of file [sci9.c](#).

```
00089 {
00090     return (volatile uint8_t *) (k_sci9_base + 4U);
00091 }
```

References [k_sci9_base](#).

Referenced by [sci9_debug_putc\(\)](#).

Here is the caller graph for this function:



4.7.2.12 sci9_tdr()

```
volatile uint8_t * sci9_tdr (
    void ) [inline], [static]
```

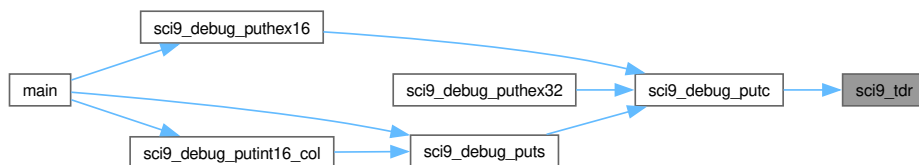
Definition at line 84 of file [sci9.c](#).

```
00085 {
00086     return (volatile uint8_t *) (k_sci9_base + 3U);
00087 }
```

References [k_sci9_base](#).

Referenced by [sci9_debug_putc\(\)](#).

Here is the caller graph for this function:



4.8 sci9.c

[Go to the documentation of this file.](#)

```

00001 /**
00002  * @file sci9_debug.c
00003  * @brief Minimal polled-mode SCI9 (TXD9 = PB7, RXD9 = PB6) debug UART for imu_test.
00004  *
00005  * @details
00006  * The STAR board routes SCI9 to the on-board CY7C65213 USB-to-UART bridge
00007  * which appears as /dev/ttyACM0 on the host (or COMx on Windows). This
00008  * file provides just enough init + putc/puts to emit ASCII status from
00009  * the firmware so we can debug the USB CDC bring-up without needing the
00010  * USB device to actually enumerate.
00011  *
00012  * Uses the rx_hal struct accessors (mpc(), portb(), system_regs(), prcr_reg())
00013  * so the addresses are all compiler-verified via static_assert.
00014  *
00015  * Baud = 115200. SCI9 is in the SCI7-11 group that uses PCLKA (120 MHz),
00016  * not PCLKB. At CKS=0, ABCS=0:
00017  * BRR = (PCLKA / (32 * baud)) - 1 = 120000000 / 3686400 - 1 = 31.55 -> 31
00018  * Actual baud = PCLKA / (32 * (BRR+1)) = 117187 Hz (+1.7% vs 115200, in tolerance).
00019  *
00020  * SPDX-License-Identifier: MIT
00021  * @copyright Copyright (c) 2026 Locked Inc.
00022  */
00023
00024 #include <stdint.h>
00025
00026 #include "rx72n_regs.h"
00027 #include "rx_register_protection.h"
00028
00029 /*
=====
00030  * SCI9 register access via raw addresses (rx72n_sci_regs.h doesn't expose
00031  * the per-channel struct as cleanly as we'd like for a quick polled driver).
00032  * The base 0x000D0020 is taken from the verified header.
00033  * Register offsets per RX72N HW manual: SMR=0, BRR=1, SCR=2, TDR=3, SSR=4.
00034  *
=====
*/
00035 typedef enum : uintptr_t {
00036     k_sci9_base = 0x000D0020U,
00037 } sci9_addrs_t;
00038
00039 typedef enum : uint8_t {
00040     k_psel_sci9      = 0x0AU, /**< SCI9 RXD/TXD function */
00041     k_pb6_mask       = (uint8_t)(1U < 6U),
00042     k_pb7_mask       = (uint8_t)(1U < 7U),
00043     k_mstpc_sci9_bit = 26U, /** MSTPCRC.MSTPC26 per RX72N HW manual page 410 */
00044     k_pwpr_unlock_b0 = 0x00U,
00045     k_pwpr_pfswe     = 0x40U,
00046     k_pwpr_b0wi      = 0x80U,
00047     k_brr_115200     = 31U, /** SCI9 uses PCLKA=120 MHz (SCI7-11 group); BRR=31 @ CKS=0 ABCS=0 -> ~117
kbaud (1.7% over 115200) */
00048     k_smr_8n1_async  = 0x00U,
00049     k_scr_te_only    = 0x20U, /**< CKE=00, MPIE=0, RIE=0, TE=1, RE=0 */
00050     k_scr_txrx_enabled = 0x30U, /**< CKE=00, MPIE=0, RIE=0, TE=1, RE=1 */
00051     k_scr_disabled    = 0x00U,
00052     k_ssr_tdre        = 0x80U,
00053     k_scmr_default    = 0xF2U,
00054     k_semr_default    = 0x00U,
00055     k_hex_nibble_mask = 0x0FU,
00056 } sci9_cfg_t;
00057
00058 typedef enum : uint16_t {
00059     k_brr_settle_loops = 1000U, /**< Match production uart.c (~8.68 us @ 115200) */
00060 } sci9_settle_t;
00061
00062 typedef enum : uint8_t {
00063     k_hex_shift_init32 = 28U,
00064     k_hex_shift_init16 = 12U,
00065     k_hex_shift_step   = 4U,
00066 } hex_shifts_t;
00067
00068 /*
=====
00069  * SCI9 register accessors (raw addresses since the header doesn't give us a
00070  * per-channel struct for the SCIi/SCIj quirks).
00071  *
=====
*/
00072 static inline volatile uint8_t *sci9_smr(void)
00073 {
00074     return (volatile uint8_t *) (k_sci9_base + 0U);
00075 }

```

```

00076 static inline volatile uint8_t *sci9_brr(void)
00077 {
00078     return (volatile uint8_t *) (k_sci9_base + 1U);
00079 }
00080 static inline volatile uint8_t *sci9_scr(void)
00081 {
00082     return (volatile uint8_t *) (k_sci9_base + 2U);
00083 }
00084 static inline volatile uint8_t *sci9_tdr(void)
00085 {
00086     return (volatile uint8_t *) (k_sci9_base + 3U);
00087 }
00088 static inline volatile uint8_t *sci9_ssr(void)
00089 {
00090     return (volatile uint8_t *) (k_sci9_base + 4U);
00091 }
00092 static inline volatile uint8_t *sci9_scmr(void)
00093 {
00094     return (volatile uint8_t *) (k_sci9_base + 6U);
00095 }
00096 static inline volatile uint8_t *sci9_semr(void)
00097 {
00098     return (volatile uint8_t *) (k_sci9_base + 7U);
00099 }
00100
00101 /*
=====
00102 * Initialise SCI9 for 115200 8N1 polled output on PB7 (TXD9).
00103 *
=====
*/
00104 void sci9_debug_init(void)
00105 {
00106     /* Step 1: release SCI9 from module stop. SCI9 = MSTPCRC bit 26 per
00107      * RX72N HW manual page 410 (NOT MSTPCRB.bit22 -- that's PDC). */
00108     *prcr_reg() = k_rx_prcr_unlock_prc0_prc1;
00109     system_regs()->mstpcrc &= ~(uint32_t)(1UL « (uint32_t)k_mstpc_sci9_bit);
00110     *prcr_reg() = k_rx_prcr_lock;
00111
00112     /* Step 2: configure PB6 (RXD9) and PB7 (TXD9) via MPC. Use the struct
00113      * accessor so the compiler computes the correct PFS offsets via
00114      * static_assert-checked layout. */
00115     mpc()->pwpr = k_pwpr_unlock_b0; /* clear B0WI */
00116     mpc()->pwpr = k_pwpr_pfswe; /* allow PFS writes */
00117     mpc()->pb6pfs = k_psel_sci9;
00118     mpc()->pb7pfs = k_psel_sci9;
00119     mpc()->pwpr = k_pwpr_unlock_b0; /* clear PFSWE */
00120     mpc()->pwpr = k_pwpr_b0wi; /* re-protect */
00121
00122     /* Step 3: PDR before PMR (production uart.c does this). PB7 = output
00123      * for TX, PB6 = input for RX. Then flip PMR to peripheral mode. */
00124     portb()->pdr |= (uint8_t)k_pb7_mask;
00125     portb()->pdr &= (uint8_t)~k_pb6_mask;
00126     portb()->pmr |= (uint8_t)(k_pb6_mask | k_pb7_mask);
00127
00128     /* Step 4: SCI9 init. Match the production order:
00129      * SCR=0 -> SMR -> BRR -> settle -> SCR=TE|RE -> SEMR.
00130      * Don't touch SCMR (reset default 0xF2 is correct). */
00131     *sci9_scr() = k_scr_disabled;
00132     *sci9_smr() = k_smr_8n1_async;
00133     *sci9_brr() = k_brr_115200;
00134
00135     /* Wait roughly one bit time before enabling TX/RX. */
00136     for (volatile uint32_t i = 0U; i < (uint32_t)k_brr_settle_loops; i++) {
00137         __asm__ volatile("nop");
00138     }
00139
00140     *sci9_scr() = k_scr_txrx_enabled;
00141     *sci9_semr() = k_semr_default;
00142 }
00143
00144 /*
=====
00145 * Polled write: spin until TDRE then drop byte into TDR.
00146 *
=====
*/
00147 void sci9_debug_putc(char c)
00148 {
00149     while ((*sci9_ssr() & (uint8_t)k_ssr_tdre) == 0U) {
00150         /* spin */
00151     }
00152     *sci9_tdr() = (uint8_t)c;
00153 }
00154
00155 void sci9_debug_puts(const char *s)
00156 {

```

```

00157     if (s == (const char *)0) {
00158         return;
00159     }
00160     while (*s != '\0') {
00161         if (*s == '\n') {
00162             sci9_debug_putc('\r');
00163         }
00164         sci9_debug_putc(*s++);
00165     }
00166 }
00167
00168 void sci9_debug_puthex32(uint32_t v)
00169 {
00170     static const char hex_digits[] = "0123456789ABCDEF";
00171     sci9_debug_putc('0');
00172     sci9_debug_putc('x');
00173     for (int8_t shift = (int8_t)k_hex_shift_init32; shift >= 0;
00174          shift -= (int8_t)k_hex_shift_step) {
00175         sci9_debug_putc(hex_digits[(v » (uint8_t)shift) & (uint8_t)k_hex_nibble_mask]);
00176     }
00177 }
00178
00179 void sci9_debug_puthex16(uint16_t v)
00180 {
00181     static const char hex_digits[] = "0123456789ABCDEF";
00182     sci9_debug_putc('0');
00183     sci9_debug_putc('x');
00184     for (int8_t shift = (int8_t)k_hex_shift_init16; shift >= 0;
00185          shift -= (int8_t)k_hex_shift_step) {
00186         sci9_debug_putc(hex_digits[(v » (uint8_t)shift) & (uint8_t)k_hex_nibble_mask]);
00187     }
00188 }

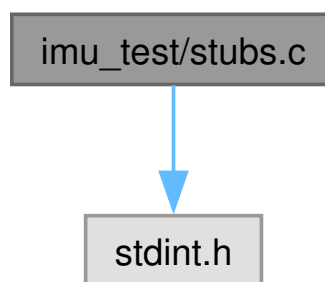
```

4.9 imu_test/stubs.c File Reference

Log/UART no-op stubs for the bare-metal IMU smoke test.

```
#include <stdint.h>
```

Include dependency graph for stubs.c:



Functions

- void [rx_log_uart_putc](#) (char c)
- void [rx_log_uart_puts](#) (const char *s)
- void [rx_log_uart_putint](#) (int32_t v)
- void [rx_log_uart_putuint](#) (uint32_t v)
- void [rx_log_uart_puthex](#) (uint32_t v, uint8_t d)
- void [uart_debug_putc](#) (char c)
- void [uart_debug_puts](#) (const char *s)
- void [uart_debug_putint](#) (int32_t v)
- void [uart_debug_putuint](#) (uint32_t v)
- void [uart_debug_puthex](#) (uint32_t v, uint8_t d)

4.9.1 Detailed Description

Log/UART no-op stubs for the bare-metal IMU smoke test.

The bench test does not pull in the rx_log family or the framed UART ring buffer. Anything in the dependency chain that calls rx_log_uart_* / uart_debug_* must have something to link against.

SPDX-License-Identifier: MIT

Copyright

Copyright (c) 2026 Locked Inc.

Definition in file [stubs.c](#).

4.9.2 Function Documentation

4.9.2.1 rx_log_uart_putc()

```
void rx_log_uart_putc (  
    char c)
```

Definition at line 16 of file [stubs.c](#).

```
00016 { (void)c; }
```

4.9.2.2 rx_log_uart_puthex()

```
void rx_log_uart_puthex (  
    uint32_t v,  
    uint8_t d)
```

Definition at line 20 of file [stubs.c](#).

```
00020 { (void)v; (void)d; }
```

4.9.2.3 rx_log_uart_putint()

```
void rx_log_uart_putint (  
    int32_t v)
```

Definition at line 18 of file [stubs.c](#).

```
00018 { (void)v; }
```

4.9.2.4 rx_log_uart_puts()

```
void rx_log_uart_puts (  
    const char * s)
```

Definition at line 17 of file [stubs.c](#).

```
00017 { (void)s; }
```

4.9.2.5 rx_log_uart_putuint()

```
void rx_log_uart_putuint (  
    uint32_t v)
```

Definition at line 19 of file [stubs.c](#).

```
00019 { (void)v; }
```

4.9.2.6 uart_debug_putc()

```
void uart_debug_putc (  
    char c)
```

Definition at line 22 of file [stubs.c](#).

```
00022 { (void)c; }
```

4.9.2.7 uart_debug_puthex()

```
void uart_debug_puthex (  
    uint32_t v,  
    uint8_t d)
```

Definition at line 26 of file [stubs.c](#).

```
00026 { (void)v; (void)d; }
```

4.9.2.8 uart_debug_putint()

```
void uart_debug_putint (  
    int32_t v)
```

Definition at line 24 of file [stubs.c](#).

```
00024 { (void)v; }
```

4.9.2.9 uart_debug_puts()

```
void uart_debug_puts (  
    const char * s)
```

Definition at line 23 of file [stubs.c](#).

```
00023 { (void)s; }
```

4.9.2.10 uart_debug_putuint()

```
void uart_debug_putuint (  
    uint32_t v)
```

Definition at line 25 of file [stubs.c](#).

```
00025 { (void)v; }
```

4.10 stubs.c

[Go to the documentation of this file.](#)

```
00001 /**
00002  * @file stubs.c
00003  * @brief Log/UART no-op stubs for the bare-metal IMU smoke test.
00004  *
00005  * @details
00006  * The bench test does not pull in the rx_log family or the framed UART
00007  * ring buffer. Anything in the dependency chain that calls
00008  * rx_log_uart_* / uart_debug_* must have something to link against.
00009  *
00010  * SPDX-License-Identifier: MIT
00011  * @copyright Copyright (c) 2026 Locked Inc.
00012  */
00013
00014 #include <stdint.h>
00015
00016 void rx_log_uart_putc(char c)      { (void)c; }
00017 void rx_log_uart_puts(const char *s) { (void)s; }
00018 void rx_log_uart_putint(int32_t v)  { (void)v; }
00019 void rx_log_uart_putuint(uint32_t v) { (void)v; }
00020 void rx_log_uart_puthex(uint32_t v, uint8_t d) { (void)v; (void)d; }
00021
00022 void uart_debug_putc(char c)      { (void)c; }
00023 void uart_debug_puts(const char *s) { (void)s; }
00024 void uart_debug_putint(int32_t v)  { (void)v; }
00025 void uart_debug_putuint(uint32_t v) { (void)v; }
00026 void uart_debug_puthex(uint32_t v, uint8_t d) { (void)v; (void)d; }
```

